

# Interpretable Fine-Grained ABSA of Public-Sector App Reviews with Integrated Gradients and LLM Justifications.

Taksh Girdhar, Ifeoma Adaji

The University of British Columbia, Okanagan Campus, BC, Canada

## Abstract.

Digital public services increasingly rely on mobile applications as primary interfaces for public interaction, yet user satisfaction with these systems remain modest. User reviews provide a rich source of feedback, however, extracting actionable insight at scale is challenging due to their noisy, multi-aspect nature and the limited interpretability of modern sentiment models. This study addresses the discrepancy by proposing an end-to-end framework for fine-grained, interpretable aspect-based sentiment analysis (ABSA) of public-sector mobile application reviews. A DistilRoBERTa-based ABSA model is fine-tuned on the FABSA benchmark, and its generalization is evaluated under domain shift to real-world government app reviews from across North America. To ensure transparency, a layered explainability pipeline is incorporated that combines Integrated Gradients (IG) for token-level attribution with LLM-generated natural-language justifications grounded explicitly in attribution evidence. Explanation quality is assessed using a set of quantitative metrics, including comprehensiveness, sufficiency, sparsity, monotonicity, and local sensitivity, adapted for text-based ABSA. Empirical results demonstrate strong predictive performance on the FABSA test set and coherent aspect-level sentiment patterns across government applications. The explainability analysis shows stable, behaviorally consistent explanations. These findings illustrate that combining fine-grained sentiment modeling with empirically evaluated explanations enables trustworthy, actionable analysis of public-sector user feedback, supporting more transparent and evidence-driven digital service improvement.

**Keywords:** Aspect-Based Sentiment Analysis (ABSA), Explainable Artificial Intelligence (XAI), Integrated Gradients (IG), Transformer Models, Public-Sector Mobile Applications, Digital Government Services, Model Interpretability.

## 1 Introduction

Digital public services now constitute one of the most significant interaction points between the public and governments worldwide, with mobile applications increasingly serving as commonly used interface for accessing health records, immigration systems, taxation platforms, transportation services, environmental alerts, and emergency communication. The global trajectory toward digital governance is reflected in the 2024

United Nations E-Government Development Index (EGDI), which reports a world average score of 0.63, with the Americas at 0.67 and Europe at 0.84 indicating that strong institutional investment but divergent digital development (Statista, 2024). Demand-side expectations are rising even more rapidly, substantiated by a 2024 national survey showing that 70% of Canadian already expect fully digital public services, and 87% expect this by 2026, signaling a fundamental shift in how the public anticipate interacting with the government (Nortal, 2024).

Despite this momentum, public satisfaction with digital government services remains modest and stagnant. U.S. customer satisfaction scores of e-government websites plateau around 75%-76% between 2013 and 2020, with limited yearly improvement (U.S. Customer Satisfaction with E-Government 2020 Statista, 2021). Even for leading federal sites such as Medicare, Social Security, and NASA portals, scores rarely exceed the low 90s (Highest-Rated U.S. Government Websites 2020 Statista, 2021). Mobile government services, which are increasingly critical for low-friction access, show a similar pattern where U.S. satisfaction scores for mobile e-government apps fluctuate in the mid-70s, with no clear upward trajectory (U.S. Customer Satisfaction with Mobile E-Government 2020 Statista, 2021). These figures contrast sharply with the public’s expectations for convenience, usability and responsiveness, particularly as private-sector digital services continue to set higher user-experience standards.

This gap highlights a structural problem. Unlike commercial platforms, public-sector mobile applications do not operate under competitive market pressures that force continuous iteration and responsiveness to user feedback. A for-profit app must integrate user reviews into rapid development cycles to maintain relevance, retention and revenue which is an ecosystem where sentiment analysis tools are widely deployed for real-time understanding of user concerns. By contrast, government apps remain essential services regardless of usability, meaning that suboptimal interfaces, reliability issues, and frustrating user experiences persist without the same organizational incentives for improvement. As prior studies of mobile government reviews have shown, users complaints frequently center on authentication failures, navigation difficulties, lack of transparency, bureaucratic delays, and poor technical features. Yet, these issues are rarely analyzed systematically at scale.

User reviews are an abundant and rich signal of public experience, but extracting actionable insights remains challenging due to their informality, noise, and multi-aspect nature. Early sentiment-analysis methods, including lexicon-based models and shallow classifiers such as SVMs or Naïve Bayes, performed adequately for coarse document-level polarity tasks but routinely failed to capture the fine-grained, aspect-specific nuances found in mobile-app reviews. These limitations stem from their reliance on hand-crafted features, domain-specific lexicons, and weak handling of contextual interpretation, making them not suitable for the noisy, informal, and multi-topic user feedback. Transformer-based models, most notably, BERT, RoBERTa and DistilRoBERTa transformed the field by introducing deep contextual representations and bi-directional self-attention, enabling more accurate modelling of implicit sentiment cues, multi-aspect text, and long-range dependencies. Recent evaluations in app review classification (UI-

lah et al., 2023; Shah et al., 2024) confirm that transformer models consistently outperform traditional baselines and often rival zero-shot performance from modern large language models (LLMs).

This shift toward aspect-based sentiment analysis (ABSA) further reflects the need to differentiate opinions about specific features such as usability, account management, staff support, or pricing rather than interpreting a review as a single polarity label. Feedback aspect-based sentiment analysis (FABSA) introduced by Kontonatsios et al. (2023), represents a major advancement in this direction. It provides over 10,00 reviews and 20,000 sentences across 12 aspect categories and three sentiment labels. Their work highlights both the necessity and difficulty of fine-grained modelling in real-world domains, particularly given the multi-aspect nature of the reviews and the substantial class imbalance inherent to user feedback. Although transformer ABSA models trained on FABSA achieve strong results, the literature also acknowledges persistent challenges, including implicit aspect cues, overlapping sentiments within the same review, and degraded performance under domain shift (for example, from commercial apps to government services).

A second challenge concerns interpretability. Despite their impressive predictive performance, transformer-based ABSA models remain opaque, making it difficult for researchers, policymakers and developers to understand why a sentiment label was assigned. Existing interpretability such as LIME, SHAP, attention-based rationales, and Integrated Gradients (IG) provide partial visibility, but they differ in stability and faithfulness, and many have never been systematically evaluated in ABSA contexts. Furthermore, while LLMs are now used to generate natural-language rationales, empirical studies warn that these explanations can be inconsistent or hallucinated unless grounded in model-internal evidence. For high-stakes public-sector applications, where misinterpretation of user complaints risks undermining trust, delaying fixes, misallocating resources, transparent and behaviorally reliable explainability is essential.

## 1.1 Problem Statement

Consequently, there is a critical need for automated, fine-grained, and interpretable analysis of government app reviews, especially one that combines state-of-the-art ABSA modelling with explanation methods capable of supporting operational decision-making in the public sector. However, no existing work systematically addresses three interrelated gaps, (i) the lack of domain-specific ABSA models evaluated on government-service applications, (ii) the absence of transparent, behaviorally reliable explanations for aspect-level predictions, and (iii) the limited integration of quantitative XAI metrics to assess the explanation quality in real-world public-sector use cases. This gap constrains the ability of government developers to realize user feedback analytics at scale and undermines trust in automated decision-support systems. To address these gaps, this study investigates the following research questions.

## 1.2 Research Questions (RQs)

1. How well does a transformer-based aspect-based sentiment classifier trained on the FABSA dataset generalize to user reviews from North American government mobile applications?
2. To what extent can Integrated Gradients and large language model generated rationales provide faithful, human interpretable explanations for aspect-level sentiment predictions?
3. How do these explanations behave under quantitative explanation-quality metrics and what do these behaviors reveal about the reliability of the explanations?

## 1.3 Objectives

Building on these research questions, the study pursues the following objectives.

1. Develop and fine-tune a transformer-based ABSA model using the FABSA benchmark and evaluate its generalization to government app reviews under real-world domain shift.
2. Incorporate an interpretable explainability pipeline that combines gradient-based attribution (Integrated Gradients) with LLM-generated textual justifications grounded in attribution evidence.
3. Design a compute a set of quantitative XAI metrics to evaluate the behavioral quality, robustness, and informativeness of the generated explanations.
4. Demonstrate practical applicability by analyzing sentiment trends and explanation outputs across widely used government mobile applications in North America.

## 1.4 Contributions

This study makes four primary contributions to the literature on sentiment analysis and digital government analytics.

1. A domain-shift evaluation of FABSA trained transformers applied to a large collection of government-service mobile application reviews. A setting rarely studied despite the rapid expansion of digital public services.
2. A hybrid XAI framework that unifies Integrated Gradients with attribution-grounded LLM justifications, producing suitable explanations for non-technical stakeholders.
3. A systematic behavioral evaluation of explanations through five quantitative metrics adapted from XAI literature, providing empirical insights into explanation faithfulness and stability.
4. A reproducible pipeline that can support future research in public-sector improvement, user feedback analytics, and interpretable ABSA.

## 2 Literature Review

Sentiment analysis is broadly concerned with the identifying the subjective orientation expressed in text, ranging from coarse document-level polarity to fine-grained analyses that capture opinion targets and contextual nuance. The shift from document and sentence level classification toward aspect-based sentiment analysis (ABSA) has been driven by the need to differentiate user opinions about specific features of product and services. In ABSA, an opinion is commonly represented as a structured tuple, the expressed sentiment, the opinion holder, and the context. This decomposition enables a more granular understanding of user feedback, particularly in domains such as mobile applications, where users frequently express conflicting attitudes across different features within the same review.

Early approaches to ABSA relied heavily on lexicon driven or shallow machine learning pipelines. Lexicon-based systems, such as those leveraging SentiWordNet or curated polarity dictionaries, matched sentiment related words to predefined polarity scores, sometime augmented with rule-based handling of negation or intensification. Although computationally lightweight, these methods struggled with domain adaptations, and nuances (for example, sarcasm) which are frequent characteristics of mobile app reviews. Similarly, traditional supervised models such as support vector machines, Naïve Bayes, and Maximum Entropy classifiers achieved moderate success when coupled with TF-IDF features or part-of-speech patterns, but their performance depended heavily on customized features and substantial annotation efforts. As mobile application reviews shifted towards highly informal, noisy, and context-dependent language, these classical techniques increasingly underperformed compared to deep contextual models.

The emergence of transformer-based architecture, particularly BERT, marked a turning point in sentiment analysis. BERT’s bidirectional self-attention mechanism and large-scale pretraining enabled models to capture semantic nuance and context-sensitive word meaning (Devlin et al., 2019). Building on these foundations, domain specific models such as DeBERTa, with its disentangled attention and enhanced mask decoder, further improved performance on classification tasks. In the domain of app reviews, Ullah et al. (2023) demonstrated that BERT-based classifiers substantially outperform conventional machine learning methods for predicting sentiment polarity at the review level, echoing similar findings across mobile banking, e-commerce, and government service review datasets. More recent evaluations comparing large language models (LLMs) with fine-tuned transformer baselines for app-review sentiment classification indicate that while LLMs offer strong zero-shot capability, domain tuned transformers such as RoBERTa and DistilRoBERTa remain competitive and substantially more efficient for large scale inference.

Within ABSA specifically, recent work has emphasized fine-grained, transformer-based aspect extraction and sentiment classification. Kontonatsios et al. (2023) introduced FABSA, a large-scale dataset of mobile app reviews annotated at the aspect-level, enabling more rigorous evaluation of fine-grained sentiment models. Their study highlighted the difficulty of modelling diverse domains and the importance of capturing implicit as well as explicit sentiment cues. Complementary research has extended this

line of inquiry to applications such as mobile banking, public-sector digital services, and healthcare platforms, illustrating both the generality and limitations of the current ABSA models. These models consistently demonstrate high performance explicit aspect-sentiment pairs but face challenges handling long-range dependencies, multi-aspect reviews, and domain shift across apps or platforms.

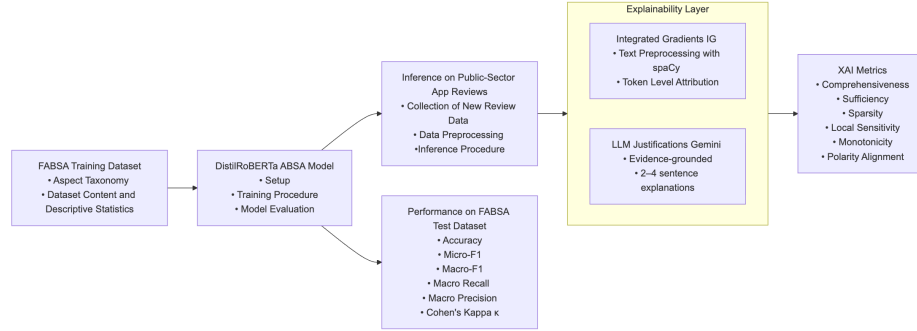
A parallel line of work focuses on interpretable and explainable sentiment analysis, motivated by the need to make transformer models more transparent to auditors, developers, and policy makers. Techniques such as Integrated Gradients, LIME, SHAP, and attention-based rationales have been widely adopted to attribute predictions to input tokens. However, these methods exhibit varying degrees of stability and faithfulness prompting the development of evaluation frameworks that quantify explanation quality metrics such as comprehensiveness, sufficiency, monotonicity, sparsity, and local sensitivity. DeYoung et al.’s (2019) ERASER benchmark, for example, formalized the idea of a rationale faithfulness, though such frameworks remain underutilized in ABSA contexts, particularly for mobile app reviews where sentiment cues may be implicit or dispersed across long text sequences.

Most recently, researchers have incorporated LLMs into ABSA pipelines either as standalone classifiers, as few-shot annotators, or as generators of natural-language justifications accompanying model predictions. Studies such as the one conducted by Shah et al. (2024) report that LLMs perform competitively with fine-tuned transformers on many ABSA tasks but often produce inconsistent or hallucinated explanations, underscoring the need for structured, attribution grounded interpretability methods. Meanwhile work on emotion extraction has shown that user feedback contains richer information than categorical sentiment alone, which further motivates the use of fine-grained app-review datasets and robust interpretability tools (Martens et al., 2017).

Despite advances in data resources, model architectures, and interpretability techniques, several critical limitations remain. Domain shift continues to degrade the performance of even the strongest transformer models when applied to new app categories. Reviews frequently contain multiple intertwined aspects, implicit sentiment, and mixed attitudes all of which are conditions in which standard ABSA pipelines may misidentify targets or misattribute polarity. Moreover, the opacity of transformer models poses challenges for deployment in high-stakes public-sector contexts, where decisions may impact service accessibility or trust. These gaps motivate the present study, which integrated the fine-tuned DistilRoBERTa-based ABSA with attribution grounded explainability and LLM-generated justifications, evaluated across both benchmark FABSA data and real-world public-sector application reviews.

### 3 Methodology

The methodology of this study integrates supervised transformer-based aspect-level sentiment modelling with a multi-layered explainability framework in order to produce both accurate and interpretable analyses of public-sector mobile applications reviews. Building on the FABSA dataset by Kontonatsios et al. (2023), the workflow begins with the construction of dense (review, aspect) training pairs that enable fine-grained sentiment classification across twelve aspects and three sentiment categories. A DistilRoBERTa encoder is then fine-tuned to learn aspect-specific polarity signals under conditions of class imbalance and domain diversity, as detailed in the training and modeling setup in Section 3.2. After deriving predictions for newly collected North American government mobile application reviews, the pipeline incorporates an explainable AI (XAI) layer that combines gradient-based attributions using Integrated Gradients (IG) with attribution-grounded LLM justifications. Finally, a set of quantitative explanation-quality metrics, adapted from the xai\_evals framework, evaluates the behavioral fidelity, robustness, and semantic coherence of the explanations. Together, these components form a reproducible end-to-end process<sup>1</sup> that supports both large-scale sentiment mapping and transparent interpretability suitable for improvement.



**Fig. 1.** End-to-end processing pipeline for interpretable aspect-based sentiment analysis of public-sector mobile application reviews.

#### 3.1 Training Dataset: FABSA (Feedback Aspect-Based Sentiment Analysis)

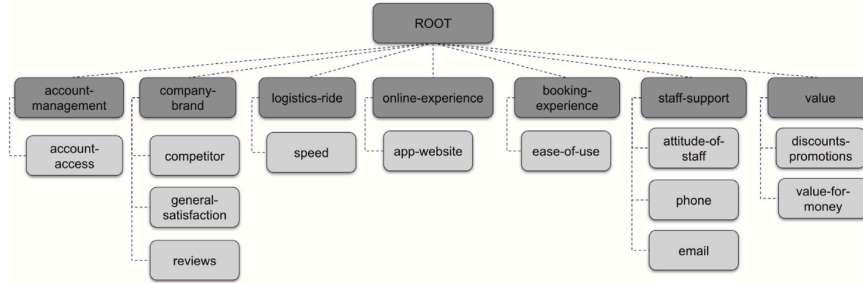
This study uses the FABSA dataset as the primary training source for building the aspect-based sentiment model. FABSA is a large-scale, manually annotated, and multi-domain aspect-based sentiment analysis (ABSA) dataset that consists of 10,574 user reviews and over 20,000 sentences. These reviews were collected from the Apple App

<sup>1</sup> To support reproducibility, all source code is publicly available at: <https://github.com/Takshg/Sentiment-Analysis-Research-Project/tree/main>

Store, the Google Play Store, and Trustpilot spanning 10 industries and 14 organizations. The rationale for adopting this dataset lies in its extensive domain coverage, enabling the sentiment model to generalize across varied types of feedback.

FABSA was created to overcome a key limitation in current ABSA resources, namely that most prior datasets are domain-restricted and relatively small, limiting the performance of transformer-based models that require large-scale supervision. Conversely, FABSA introduces 12 aspect categories, each of which are annotated with one of three sentiment labels (positive, negative, and neutral). The dataset utilizes a multi-label classification scheme, meaning that each review can encompass multiple aspects and express different sentiments toward each of them.

**Aspect Taxonomy.** The FABSA annotation scheme<sup>2</sup> consists of 7 parent aspects and 12 child aspects, forming the full set of aspect categories that the sentiment model must learn to classify.



**Fig. 2.** Annotation scheme of the FABSA dataset as provided by Kontonatsios et al. (2023).

1. account-management.account-access  
Reviews that address issues concerning logging in, signing up, forgotten usernames/passwords, identity verification, or personal details (for example, contact information, addresses, identification documents). Some examples are, “couldn’t get into my account”, “forgot my username”, “registration problems”.
2. company-brand.competitor  
Reviews that mention competitor products or make comparisons with other companies (for example, “cheaper than others” or “better than X”). Additionally, evaluative phrases such as “best”, “favorite” or “worst” that implicitly reference competitors are included.
3. company-brand.general-satisfaction  
Reviews that are general statements about overall satisfaction or dissatisfaction with the brand or service. This aspect includes broad evaluative comments such as “great service”, “love it”, “awful”, or “never using them again”.
4. company-brand.reviews

<sup>2</sup> <https://github.com/kontonag86/fabsa-dataset/blob/main/FABSA - Annotation Guidelines.pdf>



Reviews that mention ratings, online discussions, or a general review related to the organization, including references to other users' reviews, forums, and discussion on online platforms hosted by the organization.

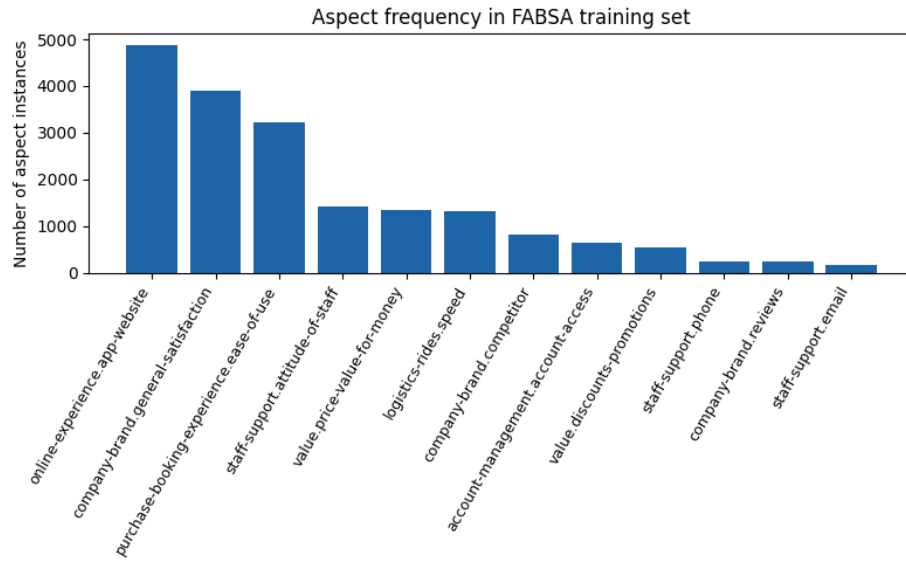
5. logistics-ride.speed  
Reviews that are related to the speed or timeliness of service delivery, processing, or response time of the app/website/service provided by the organization including descriptors such as "quick", "fast", or "slow".
6. online-experience.app-website  
Reviews that cover functionality, usability and stability of the app. These reviews include navigation issues, crashes, bugs, freezing, loading problems, and general usability descriptors such as "user-friendly", "confusing" or "hard to use".
7. purchase-booking-experience.ease-of-use  
Reviews that refer to the ease or difficulty of the purchasing or booking process, including checkout flow, online basket issues, and the overall transaction experience.
8. staff-support.attitude-of-staff  
Reviews that mention staff behavior or communication style, including descriptors for politeness, professionalism, friendliness, helpfulness, patience or rudeness.
9. staff-support.phone  
Reviews that specifically refer to phone-based customer support interactions. For example, "tried calling this outfit but they don't answer the phone".
10. staff-support.email  
Reviews that specifically refer to email-based customer support interactions. For example, "communicated with them via email and they responded showing a gesture of good-will which solved all my problems."
11. value.discounts-promotions  
Reviews that mention vouchers, coupons, gift cards and codes. For example, "saved money and got a free voucher, happy!" or "did get my car insurance a lot cheaper but never got my free gift".
12. value.price-value-for-money  
Reviews that relate to cost, affordability, or value received for the price paid. These reviews include descriptors such as "expensive", "cheap" or "not worth the money".

The following Table 1 demonstrates how a single FABSA review can encompass multiple aspects with contrasting sentiments, thereby highlighting the fine-grained structure that aspect-based sentiment analysis must capture.

**Table 1.** An example of a review from the training dataset containing multiple aspects with contrasting sentiments.

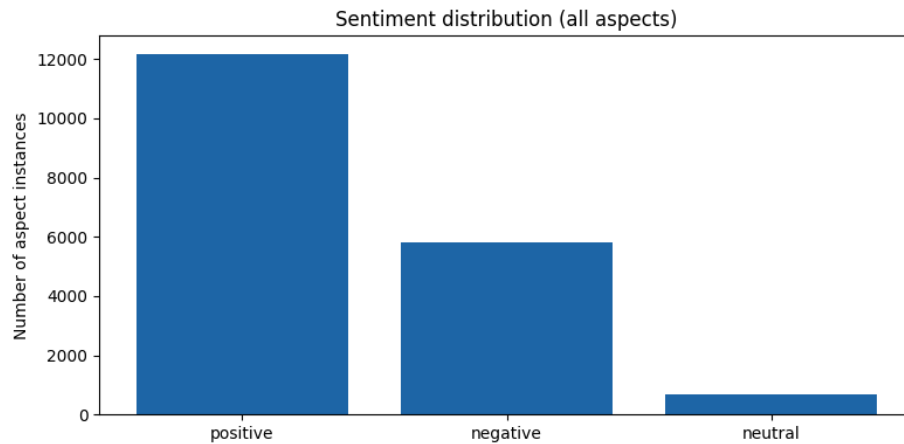
| Review                                                                       | Aspects & Sentiment                                                                 |
|------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| "Fast shipment, but one of the three ordered items is damaged - for return." | logistics-rides.speed <positive> ;<br>company-brand.general-satisfaction <negative> |

**Dataset Content and Descriptive Statistics.** A series of descriptive analyses was conducted to better understand the structure and distribution of the FABSA training dataset. These statistics provide an overview of the aspect composition of the reviews, the balance of the sentiment labels and the relative frequency of different aspect categories. These patterns are essential for assessing data quality, identifying class imbalance and understanding the challenges that arise from multi-aspect sentiment modelling.



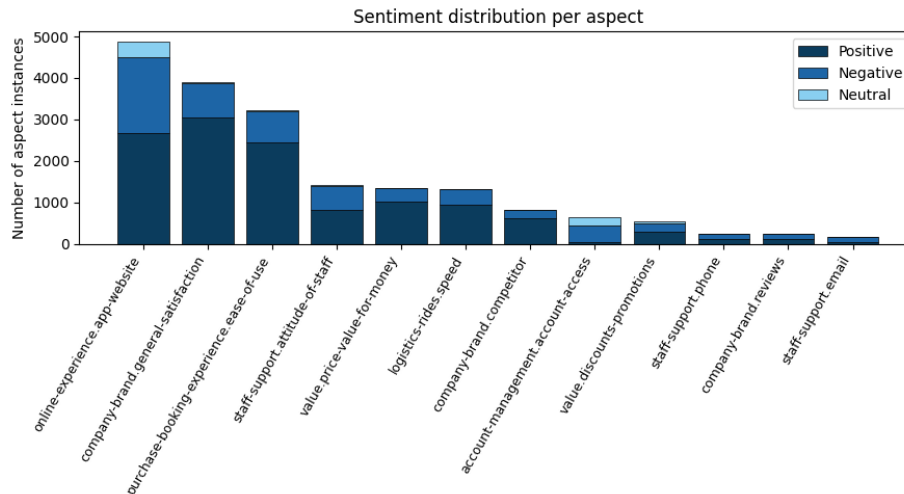
**Fig. 3.** Distribution of aspect labels in the FABSA dataset.

As shown in Figure 3, the frequency of all aspect categories across the dataset are highly skewed. While online-experience.app-website and company-brand.general-satisfaction dominate, aspects such as staff-support.email and company-brand.reviews are not as prevalent. This imbalance reflects real-world feedback trends where users comment more frequently on app/website functionality and overall satisfaction rather than niche interactions. However, this distribution highlights areas where the sentiment analysis model may need more cautious handling due to limited training samples.



**Fig. 4.** Overall sentiment labels distribution in the FABSA dataset.

The overall sentiment labels distribution combined across all aspects is shown in Figure 4. The dataset is heavily skewed in favor of positive sentiment, with a smaller percentage of negative labels and a comparatively small neutral class. Model training may be impacted by this imbalance as the sentiment analysis model could naturally tend to predict the majority sentiment unless corrective techniques are used.



**Fig. 5.** Distribution of sentiment labels per aspect in the FABSA dataset.

Lastly, Figure 5 illustrates how polarity varies according to the review's theme by breaking down sentiment by aspect category. For instance, logistics-rides.speed and staff-support.attitude-of-staff contain a higher proportion of negative sentiment compared to categories such as value.discounts-promotions or purchase-booking-experi-

ence.case-of-use which is more positively skewed. With this detailed view, the diversity within the FABSA dataset and the importance of aspect-specific modelling is highlighted. Different aspects naturally result in different sentiment patterns, and a reliable ABSA model is needed to capture the intricate relationships.

Together, these dataset statistics assist in establishing the context for subsequent modelling and evaluation stages by providing a clear picture of the distributional characteristics.

### 3.2 DistilRoBERTa ABSA Model

**Setup.** In order to construct the aspect-based sentiment classifier, a DistilRoBERTa encoder is fine-tuned in a sentence-pair classification setting. DistilRoBERTa with 82M parameters is a compressed version of RoBERTa which has 125M parameters, while being twice as fast (Distilroberta Base, 2023).

Each training instance is associated with a (review, aspect) pair. Given a review text  $r$  and an aspect label  $a$  from the FABSA schema, an input sequence of the form below is constructed

$$[CLS] \ r \ [SEP] \ a \ [SEP] \quad (1)$$

using the DistilRoBERTa tokenizer. The tokenizer then splits the text into sub-word pieces, maps them to token IDs, and produces an attention mask that indicates which positions are real tokens and which are padding.

The paired sequence is passed through the DistilRoBERTa encoder which outputs a contextual embedding for every token. Next, the hidden state at the first position (related to the special classification token) as a pooled representation  $h \in \mathbb{R}^4$  for the whole (review, aspect) pair is taken together with a linear classification head on top represented by

$$z = Wh + b \quad (2)$$

where  $W \in \mathbb{R}^{4 \times d}$  and  $b \in \mathbb{R}^4$ . The resulting logits  $z$  are passed through a softmax layer to obtain class probabilities over the four labels in Table 2 below.

**Table 2.** Definition of four output classes used by the DistilRoBERTa aspect-sentiment classifier.

| Label    | Aspect Presence    | Sentiment Polarity |
|----------|--------------------|--------------------|
| Positive | Aspect Present     | +1                 |
| Neutral  | Aspect Present     | 0                  |
| Negative | Aspect Present     | -1                 |
| Absent   | Aspect not present |                    |

Designing the problem in such a way allows the conversion from the original multi-label task into a 4-way classification problem per aspect while still enabling a reconstruction of a full set of aspect-sentiment labels during prediction.

### Training Procedure.

*Construction of training pairs.* From the FABSA training split, a dataframe `train_pairs` is constructed. Subsequently, for review  $r_i$  and each aspect  $a_j$  in the 12-aspect taxonomy, the presence of gold FABSA annotations for review  $r_i$  are checked. If  $a_j$  appears in  $r_i$  with sentiment  $s \in \{-1, 0, +1\}$ , a pair  $(r_i, a_j)$  is created with label  $s$  that is mapped to the negative, or neutral, positive labels in Table 2. If  $a_j$  does not appear in  $r_i$ , a pair  $(r_i, a_j)$  with the absent label.

This results in a dense set of training pairs of size  $N_{reviews} \times 12$ . In the notebook titled `Sentiment_Analysis_Project_FABSA_Section`<sup>3</sup>, this is implemented by iterating over the rows of the FABSA dataframe, expanding each review into 12 candidate aspects, and storing the resulting pairs and labels in a new dataframe that is then wrapped into a Pytorch Dataset.

*Optimization and Training Loop.* The model is now fine-tuned end-to-end using AdamW with a small learning rate having order of  $10^{-5}$ , a linear learning rate scheduler, and miniature batches of paired inputs. The code’s executions is during in each training step is described in the following Table 3 below.

**Table 3.** Overview of the DistilRoBERTa fine-tuning process. Each row describes one computational step executed during training.

| Step                 | Description                                                                                                                                                                                                                                                                                                                             |
|----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Tokenization         | Converts each batch of (review, aspect) strings into readable inputs using a <code>Dataset.map()</code> function. The tokenizer splits text into sub-word tokens, maps them to vocabulary IDs ( <code>input_ids</code> ), and generates an <code>attention_mask</code> for padded sequences.                                            |
| Forward Pass         | Feeds tokenized inputs ( <code>input_ids</code> , <code>attention_mask</code> ) during into the DistilRoBERTa encoder to obtain contextualized embeddings for every token. The pooled representation (the hidden state at the first token) is then extracted as a fixed-dimensional representation of the entire (review, aspect) pair. |
| Classification Layer | Passes the pooled embedding through a linear classification head to compute logits over the 4 output classes in Table 2. The logits signify the unnormalized evidence of model confidence for each output class.                                                                                                                        |
| Loss Computation     | Applies the softmax-cross-entropy loss between the logits and the true label (labels). The scalar loss measures the difference between the predicted and gold aspect-sentiment class and is minimized during optimization.                                                                                                              |
| Back-propagation     | Transfers error information from the output layer back through the network to guide learning by computing the gradients of the loss for all trainable parameters in the encoder and the classification head.                                                                                                                            |
| Parameter update     | Updates the optimizer (AdamW) using the calculated gradients. The learning rate and gradient accumulation (every 8 steps) are automatically by the Trainer class                                                                                                                                                                        |
| Periodic Evaluation  | Evaluates model development on the development set using the custom <code>compute_metrics</code> function on an interval of every 100 steps. The new micro-F1 score at every interval is compared with the previous. If it exceeds the previous score, the model checkpoint is updated to load the best model at the end.               |

<sup>3</sup> [https://github.com/Takshg/Sentiment-Analysis-Research-Project/blob/main/notebooks/Sentiment\\_Analysis\\_Project\\_FABSA\\_Section.ipynb](https://github.com/Takshg/Sentiment-Analysis-Research-Project/blob/main/notebooks/Sentiment_Analysis_Project_FABSA_Section.ipynb)

A pseudocode version of the training loop is given in Algorithm 1, 2 and 3 below. Although the steps in Table 3 and the algorithms are handled internally by the HuggingFace Trainer, the conceptual operations executed during each training update are reported for clarity. The HuggingFace Trainer handles the core optimization loop internally, for each batch it performs a forward pass through DistilRoBERTa, computes cross-entropy loss, runs backpropagation, updates parameters using AdamW (with gradient accumulation), and periodically evaluates on the development set.

---

**Algorithm 1** FABSA Data Preparation and Sentence–Aspect Pair Construction

---

**Require:** FABSA master dataset  $D$ ; aspect set  $A$ ; label mappings LABEL2ID; random seed  $s$

- 1: **Load/Create dataset splits**
- 2: **if** `train.csv`, `dev.csv`, `test.csv` exist **then**
- 3:     Load ( $R_{train}$ ,  $R_{dev}$ ,  $R_{test}$ )
- 4: **else**
- 5:     Load master dataset  $D$
- 6:     Derive per-review stratification key `aspect|sentiment`
- 7: **end if**
- 8: Split  $D$  into  $R_{train}$  (70%) and  $R_{temp}$  (30%)
- 9: Split  $R_{temp}$  into  $R_{dev}$  (10%) and  $R_{test}$  (20%)
- 10: Save splits
- 11: **function** MAKEPAIRS( $R$ ,  $A$ )
- 12:     Initialize empty list  $P$
- 13:     **for** each review  $r \in R$  **do**
- 14:         Parse gold labels into set of  $(a, y)$
- 15:         **for** each aspect  $a \in A$  **do**
- 16:              $y \leftarrow \text{absent}$
- 17:             **if**  $(a, y^*)$  exists **then**
- 18:                  $y \leftarrow y^*$
- 19:             **end if**
- 20:             Append (`id`, `text`,  $a$ ,  $y$ , LABEL2ID[ $y$ ]) to  $P$
- 21:         **end for**
- 22:     **end for**
- 23:     **return** dataframe  $P$
- 24: **end function**
- 25:  $P_{train} \leftarrow \text{MAKEPAIRS}(R_{train}, A)$
- 26:  $P_{dev} \leftarrow \text{MAKEPAIRS}(R_{dev}, A)$

---

---

**Algorithm 2** Training DistilRoBERTa for Aspect-Based Sentiment Classification
 

---

**Require:** Pair datasets  $P_{train}, P_{dev}$  with columns `{text, aspect, target_label_id}`; label set  $L = \{\text{negative}, \text{neutral}, \text{positive}, \text{absent}\}$ ; pretrained model ID `distilroberta-base`; random seed  $s = 88$ ; max sequence length  $M = 64$ ; learning rate  $\eta = 3 \times 10^{-5}$ ; epochs  $E = 2$ ; per-device train batch size  $B_{tr} = 8$ ; per-device eval batch size  $B_{ev} = 2$ ; gradient accumulation steps  $G = 8$ ; logging steps  $\ell = 10$ ; eval steps  $N = 100$ ; save steps  $S = 100$ ; selection metric `f1_micro`; output directories `MODEL_DIR`, `TB_ROOT`.

```

1: Initialise tokenizer and model
2:  $T \leftarrow \text{AUTOTOKENIZER.FROM\_PRETRAINED}(\text{distilroberta-base})$ 
3:  $f_\theta \leftarrow \text{AUTOMODELFORSC.FROM\_PRETRAINED}(\text{distilroberta-base})$ 
4: Enable gradient checkpointing:  $f_\theta.\text{gradient\_checkpointing\_enable}()$ 
5: Disable KV-cache for training stability:  $f_\theta.\text{config.use\_cache} \leftarrow \text{False}$ 

6: Build tokenized HuggingFace datasets
7: function BUILDHFDATASETS( $P, T, M$ )
8:   Convert to HF Dataset:  $H \leftarrow \text{DATASET.FROM\_PANDAS}(P)$ 
9:   Rename label column:  $H.\text{rename\_column}(\text{target\_label\_id}, \text{labels})$ 
10:  Define tokenizer mapping (batched):
       $(\text{text}, \text{aspect}) \mapsto T(\text{text}, \text{text\_pair}=\text{aspect})$ 
11:  Remove non-training columns (retain only token fields + labels)
12:  Set torch format:
13:   $H.\text{set\_format}(\text{type}=\text{torch}, \text{columns} = \{\text{input ids},$ 
14:     $\text{attn mask}, \text{labels}\})$  return  $H$ 
15: end function
16:  $HF_{train} \leftarrow \text{BUILDHFDATASETS}(P_{train}, T, M)$ 
17:  $HF_{dev} \leftarrow \text{BUILDHFDATASETS}(P_{dev}, T, M)$ 

18: Define evaluation metrics (HuggingFace Evaluate)
19: function COMPUTEMETRICS( $(\text{logits}, \text{labels})$ )
20:    $\text{preds} \leftarrow \arg \max(\text{logits}, \text{axis} = -1)$ 
21:   return  $\{\text{accuracy}, \text{f1\_micro}, \text{f1\_macro}, \text{precision\_macro},$ 
       $\text{recall\_macro}\}$ 
22: end function

23: Configure TensorBoard
24:  $RUN\_NAME \leftarrow \text{distilroberta-base\_FABSA\_seed}$ 
25: Create logging directory:  $RUN\_DIR \leftarrow \text{TB\_ROOT}/RUN\_NAME$ 

```

---

---

**Algorithm 3** Trainer Configuration and Optimization
 

---

```

1: Configure training arguments
2:  $\alpha \leftarrow \text{TRAINING\_ARGUMENTS}(\text{
  output\_dir} = \text{/content/drive/.../outputs/tb/RUN\_NAME},
  \text{run\_name} = \text{RUN\_NAME},
  \text{logging\_dir} = \text{RUN\_DIR},
  \text{report\_to} = [\text{tensorboard}],
  \text{logging\_strategy} = \text{steps},
  \text{logging\_steps} = \ell = 10,
  \text{learning\_rate} = \eta = 3 \times 10^{-5},
  \text{num\_train\_epochs} = E = 2,
  \text{per\_device\_train\_batch\_size} = B_{tr} = 8,
  \text{per\_device\_eval\_batch\_size} = B_{ev} = 2,
  \text{gradient\_accumulation\_steps} = G = 8,
  \text{eval\_strategy} = \text{steps},
  \text{eval\_steps} = N = 100,
  \text{save\_strategy} = \text{steps},
  \text{save\_steps} = S = 100,
  \text{load\_best\_model\_at\_end} = \text{True},
  \text{metric\_for\_best\_model} = \text{f1\_micro},
  \text{greater\_is\_better} = \text{True},
  \text{seed} = s = 88,
  \text{dataloader\_num\_workers} = 0,
  \text{dataloader\_pin\_memory} = \text{False},
  \text{disable\_tqdm} = \text{False},
  \text{use\_mps\_device} = \text{False}
})$ 
```

3: **Initialise Trainer**

```

4:  $\text{collator} \leftarrow \text{DATA\_COLLATOR\_WITH\_PADDING}(\text{tokenizer}=\text{T})$ 
5:  $\text{trainer} \leftarrow \text{TRAINER}(\text{
  model} = f_{\theta},
  \text{args} = \alpha,
  \text{train\_dataset} = HF_{train},
  \text{eval\_dataset} = HF_{dev},
  \text{tokenizer} = T,
  \text{data\_collator} = \text{collator},
  \text{compute\_metrics} = \text{COMPUTE\_METRICS}
)$ 
```

6: **Train and select best checkpoint**

```

7:  $\text{trainer.TRAIN}()$ 
8: Evaluation performed every  $N$  steps; best model selected by micro-F1
```

9: **Save model artifacts**

```

10:  $\text{trainer.SAVE\_MODEL}(\text{MODEL\_DIR})$ 
11:  $T.SAVE\_PRETRAINED(\text{MODEL\_DIR})$ 
12: TensorBoard logs stored in  $\text{RUN\_DIR}$ 
```

13: **return** fine-tuned DistilRoBERTa ABSA classifier  $f_{\theta}$  and tokenizer  $T$

---



**Model Evaluation on FABSA Test Pairs.** To evaluate the performance of the fine-tuned DistilRoBERTa model on the FABSA test split, a structured inference pipeline is summarized below and formalized in Algorithm 4.

Using the same expansion strategy in construction of training pairs, a corresponding dataframe `test_pairs` is created from the FABSA split for evaluation. Each test review is similarly paired with all 12 aspects, and each pair is assigned its gold label from the FABSA annotations. This fulfills two needs, firstly, it provides a labelled test set for calculating classification metrics at an aspect-sentiment level, and secondly, it provides a dense grid of predictions that will later be reused for explainable AI in generating token attributions for specific aspect predictions.

The evaluation process begins by expanding the raw test reviews into a full set of review-aspect pairs and converts them to into a HuggingFace dataset. Next, batched inferences is applied using the saved model checkpoint, producing logits and predicted labels for each pair. Lastly, these predictions are aggregated to compute the main aspect-sentiment classification metrics detailed below. Additionally, measures such as Cohen’s Kappa and a confusion matrix are all used to characterize model behavior across the 4-class outputs.

*Metrics.* Each combination of aspect and sentiment  $(a_j, s_k)$  is treated as an independent label, producing up to 36 labels  $(12 \text{ aspects} \times 3 \text{ sentiments})$ . Despite the model being trained as a 4-class classifier (positive, neutral, negative, absent), the “absent” class is only relevant for aspect-detection accuracy. For a classification-level evaluation, including the following metrics, “absent” is treated as a valid lass and appears in the confusion matrix because it is important to know if the classifier has learnt to recognize when an aspect does not appear in the review.

**Table 4.** TP/FP/FN for Aspect-Sentiment Evaluation

| Quantity                      | Contextual Definition                                                                                                                                                                                                                                                        |
|-------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| True Positives ( <i>TP</i> )  | Number of review-aspect pairs where the model correctly predicts the gold aspect-sentiment label. Only aspect instances that are present in the review are considered for <i>TP</i> .                                                                                        |
| False Positives ( <i>FP</i> ) | Number of review-aspect pairs where the model predict a given aspect-sentiment label, but the gold label corresponds to a different sentiment. These include cases where the model assigns a sentiment to an aspect that was actually annotated as absent.                   |
| False Negatives ( <i>FN</i> ) | Number of review aspect pairs where the gold aspect-sentiment label is present (negative, neutral, positive), but the model predicts a different label. This also includes cases where the model predicts absent for an aspect that truly expresses sentiment in the review. |

$$\text{Accuracy} = \frac{TP+TN}{P+N} \quad (3)$$

$$\text{Macro Recall} = \frac{1}{n} \sum_{i=1}^n \frac{TP_i}{TP_i+FN_i} \quad (4)$$

$$\text{Macro Precision} = \frac{1}{n} \sum_{i=1}^n \frac{TP_i}{TP_i + FP_i} \quad (5)$$

$$\text{F1-Score} = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (6)$$

$$\text{Macro F1-Score} = \frac{1}{n} \sum_{i=1}^n \text{F1-Score}_i \quad (7)$$

$$\text{Micro F1-Score} = \frac{\sum_{i=1}^n M_{ii}}{\sum_{i=1}^n M_{ii} + \frac{1}{2}[(\sum_{i=1}^n \sum_{j=1; i \neq j}^n M_{ij}) + (\sum_{i=1}^n \sum_{j=1; i \neq j}^n M_{ji})]} \quad (8)$$

where  $M_{ij}$  indicates the element for the  $i$ th row and  $j$ th column in a confusion matrix.

$$\text{Cohen's Kappa } \kappa = \frac{P_o - P_e}{1 - P_e} \quad (9)$$

where  $P_o = \frac{1}{N} \sum_{i=1}^K C_{ii}$  which is the sum of the diagonal of the confusion matrix and  $P_e = \frac{1}{N^2} \sum_{i=1}^K [\sum_{j=1}^K C_{ij} \cdot \sum_{j=1}^K C_{ji}]$  which represents how often the model and gold labels would agree if labels were assigned according to their marginal distribution.

Cohen's Kappa is a complementary metric that has been especially added to the performance evaluation because it provides a chance-corrected agreement between the model's predicted class labels and the gold labels. Contrasting with simple accuracy, Cohen's Kappa accounts for the fact that the model could achieve non-trivial agreement by randomly predicting the majority class. This is particularly a relevant issue when using the FABSA dataset because the distribution of sentiment labels are imbalanced as shown in Figure 4.

Evaluation metrics follow standard definitions used in contemporary machine learning literature (McHugh ML, 2025; Géron, 2019; Jurafsky & Martin, 2024). These metrics allow the analysis of how performance varies between frequent aspects such as online-experience.app-website and rare aspects such as staff-support.email. The numerical values are presented in the Results section.

---

**Algorithm 4** FABSA Test-Set Evaluation and Metric Computation

---

**Require:** FABSA test split  $R_{\text{test}}$ ; aspect set  $A = \{a_1, \dots, a_{12}\}$ ; model  $f_\theta$ ; tokenizer  $T$ ; evaluation batch size  $B_{ev}$ ; label mappings ID2LABEL

```

1: Expand reviews into sentence–aspect pairs
2:  $P_{\text{test}} \leftarrow \emptyset$ 
3: for each review  $r \in R_{\text{test}}$  do
4:   for each aspect  $a \in A$  do
5:     Assign gold label  $y \in \{\text{negative, neutral, positive, absent}\}$ 
6:     Append  $(\text{id}, \text{text}, a, y, \text{LABEL2ID}[y])$  to  $P_{\text{test}}$ 
7:   end for
8: end for

9: Tokenise sentence–aspect pairs
10: function TOKENIZE( $P_{\text{test}}$ )
11:   Apply  $T$  to each pair:
12:      $(\text{text}, \text{aspect}) \mapsto (\text{input\_ids}, \text{attn\_mask})$ 
13:   return tokenised dataset
14: end function
15:  $HF_{\text{test}} \leftarrow \text{TOKENIZE}(P_{\text{test}})$ 
16: Set tensor format with fields  $\{\text{input\_ids}, \text{attn\_mask}, \text{labels}\}$ 

17: Initialise evaluation dataloader
18:  $\text{collator} \leftarrow \text{DATACOLLATORWITHPADDING}(T)$ 
19:  $\text{loader} \leftarrow \text{DATA LOADER}(\text{dataset} = HF_{\text{test}}, \text{batch\_size} = B_{ev}, \text{shuffle} = \text{False}, \text{collate\_fn} = \text{collator})$ 

20: Run inference
21: Move  $f_\theta$  to device and set  $f_\theta$  to evaluation mode
22: Disable gradient computation
23: Initialise empty lists  $L_{\text{logits}}, L_{\text{labels}}$ 
24: for each batch  $b$  in  $\text{loader}$  do
25:   Extract labels  $y_b$  and move inputs to device
26:    $\ell_b \leftarrow f_\theta(b). \text{logits}$ 
27:   Append  $\ell_b$  to  $L_{\text{logits}}$ 
28:   Append  $y_b$  to  $L_{\text{labels}}$ 
29: end for

30: Aggregate predictions
31:  $Z \leftarrow \text{CONCAT}(L_{\text{logits}})$ 
32:  $Y \leftarrow \text{CONCAT}(L_{\text{labels}})$ 
33:  $\hat{Y} \leftarrow \arg \max(Z, \text{axis} = -1)$ 

34: Compute evaluation metrics
35: Compute micro-F1, macro-F1, accuracy, macro-precision, macro-recall
36:  $\kappa \leftarrow \text{COHENKAPPA}(Y, \hat{Y})$ 
37:  $CM \leftarrow \text{CONFUSIONMATRIX}(Y, \hat{Y})$ 

38: return classification metrics, Cohen’s  $\kappa$ , and confusion matrix  $CM$ 

```

---

### 3.3 Inference using DistilRoBERTa ABSA Model

**Collection of New Review Data.** In order to assess the applicability of the proposed DistilRoBERTa aspect-based sentiment analysis model to the real-world public sector services, new and independent user reviews from mobile applications developed by federal, state, provincial government agencies across North America (Canada and United States of America) were collected. Digital identity platforms, tax-service applications, health service portals, transportation tools, are just a few examples of the vital digital infrastructures that the public use to engage with government services. Given that these applications' user experience problems have a direct impact on trust, digital inclusion, and accessibility, methodically analyzing their review data offers an evidence-based mechanism for identifying areas that need improvement.

The list of target mobile applications was compiled via a manual, exploratory search process across a range of public-facing government sources and repositories. This included federal, state, and agency-specific websites, digital service portals, and references embedded within official communications, where mobile applications were advertised as access points for public services. Rather than relying on a centralized registry, which does not currently exist for government mobile applications, the tedious selection process involved iteratively identifying apps through keyword-based searches and navigation of departmental websites, followed by verification of their presence on each store.

The identified applications were then combined into a structured reference file, `App_IDs_List.xlsx`<sup>4</sup>, which records each app alongside its unique store identifier, application name, and platform source. This reference file serves as a unifying index for downstream scraping, data integration, and model inference. User reviews were subsequently retrieved from the Google Play Store and the Apple App Store the two dominant mobile ecosystems using a custom set of automation tools for data collection, listed in Table 5.

While this selection process was intentionally inclusive, it was not exhaustive, reflecting the broader challenge that government mobile applications are often fragmented across agencies and jurisdictions, with limited visibility or centralized documentation. This lack of systematic tracking underscores the need for scalable, automated methods to continuously monitor public-sector digital services and to analyze user feedback at scale.

---

<sup>4</sup> [https://github.com/Takshg/Sentiment-Analysis-Research-Project/blob/main/app\\_scraper/App\\_IDs\\_List.xlsx](https://github.com/Takshg/Sentiment-Analysis-Research-Project/blob/main/app_scraper/App_IDs_List.xlsx)

**Table 5.** Overview of Data Collection Pipeline of New Reviews from Public Sector Applications.

| Tool                | Description                                                                                                                                                                                                                                                                                                                                       |
|---------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Single-app Scraping | Uses <code>scrape_reviews.py</code> to download review for an individual app by specifying <code>store</code> , <code>app_id</code> , <code>language</code> (in this case only EN), <code>region</code> , maximum number of reviews, and an optional <code>regex</code> filter. Each run produces a CSV file containing raw reviews and metadata. |
| Batch Scraping      | Uses <code>batch_scrape.py</code> to iterate over all entries in <code>App_IDs_List.xlsx</code> and invokes <code>scrape_reviews.py</code> for each app, producing one CSV file per application.                                                                                                                                                  |
| Batch Merging       | Uses <code>merge_batch_results.py</code> to concatenate all per-app CSVs into a single unified collection, normalizes columns names such as <code>review_id</code> , <code>app_id</code> , <code>store</code> and removes duplicate entries if present.                                                                                           |

This approach ensures a comprehensive coverage of public sector mobile applications across North America. The merged dataset `all_reviews.csv` forms the foundation for applying the DistilRoBERTa aspect-based sentiment analysis model to assess user-reported strengths, frustrations, and opportunities for improvement in public digital service delivery.

**Data Preprocessing.** Before inference, the newly collected reviews must undergo a preprocessing pipeline designed to standardize the diverse review formats returned by the two app stores. The key preprocessing steps are detailed in the Table 6 below.

**Table 6.** Preprocessing steps applied to new reviews prior to model inference

| Step                   | Description                                                                                                                                                                                                         |
|------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Schema Normalization   | Ensures all reviews are normalized into a common schema that includes <code>review_id</code> , <code>app_id</code> , <code>store</code> , <code>text</code> (review body). Cast text columns to UTF-8 strings.      |
| App Metadata Alignment | Joins the merged dataset <code>all_reviews.csv</code> with reference file <code>App_IDs_List.xlsx</code> to attach the human readable app names to each <code>app_id</code> .                                       |
| Aspect Expansion       | For each review, constructs the full set of (review, aspect) pairs using predefined aspect list. The function <code>build_pairs()</code> creates one row per (review, aspect) combination with associated metadata. |
| HF Dataset Conversion  | Convert the resulting dataframe of pairs into a HuggingFace Dataset, preparing for batched tokenization and Pytorch formatting.                                                                                     |

After preprocessing, the newly collected reviews are represented in a unified format with consistent identifiers, attached app metadata, and an explicit aspect schema. This coherent representation allows the usage of the fine-tuned DistilRoBERTa ABSA model to the same class of (review, aspect) pairs used during supervised training. Importantly, the preprocessing pipeline is intentionally light because this level of normalization is sufficient to preserve the raw linguistic signals such as typos, informal phrasing, and code-switching for the model to exploit during inference. With this structure

in place, the next stage performs batched inference over the expanded pairs, decodes aspect-sentiment predictions, and assembles model outputs suitable for further XAI analysis

**Inference Procedure on New Government App Reviews.** Using the new set of normalized reviews from the federal, state, and provincial government mobile applications in North America, the fine-tuned DistilRoBERTa aspect-based sentiment analysis model is used to generate aspect-based sentiment predictions for public sector applications. The inference pipeline follows the same structural pattern as the training and evaluation workflow, operating on (review, aspect) pairs to ensure compatibility.

The inference procedure passes the preprocessed review dataset through the fine-tuned DistilRoBERTa classifier after batched tokenization. The model outputs logits for the four class labels in Table 2, which are converted into probabilities using a numerically stable softmax. A predicted sentiment class is then obtained and mapped to human-readable labels. These predictions, together with model confidence scores, are merged back with the original metadata to produce a structured prediction table at the (review, aspect) level and will be presented in the Results section.

The full sequence of the inference procedure is summarized in Table 7 below along with a corresponding high-level algorithm provided in Algorithms 5 and 6 .

**Table 7.** Detailed steps of the inference pipeline for applying fine-tuned DistilRoBERTa model to new government app reviews.

| Step                      | Description                                                                                                                                                                                                                   |
|---------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Text field Sanitization   | Fills missing values in text and aspect with empty strings and cast them to string types. This ensures that all pairs can be safely processed by the tokenizer without raising errors due to NaN values or non-strings types. |
| Tokenizer & Model loading | Loads the saved fine-tuned DistilRoBERTa tokenizer and classification model. Set the model to evaluation mode,                                                                                                                |
| Batched Model Inference   | Iterates over the DataLoader inside a <code>torch.no_grad()</code> context. For each batch, move tensors to the device and compute logits. Accumulate all logits across batches and convert to NumPy array.                   |
| Softmax & Label decoding  | Computes class probabilities using a numerically stable softmax over logits and then derive the predicted sentiments. Extract the predicted class probability for each pair.                                                  |

---

**Algorithm 5** Inference Pipeline for Government App Reviews (Aspect Expansion)

---

**Require:** review file `input.csv` ( `all_reviews.csv`); app reference file `App_IDs_List.xlsx`; aspect set  $A = \{a_1, \dots, a_{12}\}$ ; model  $f_\theta$

```

1: Load raw reviews
2:  $R \leftarrow \text{READCSV}(\text{input\_csv})$     ▷ must have: app_id, store, review_id, text

3: Load app metadata (app_id → app_name) and attach names to reviews
4:  $M \leftarrow \text{READXCEL}(\text{App\_IDs\_List.xlsx})$ 
5: Rename columns in  $M$ : "App Name" → "app_name", "App ID" → "app_id"
6: Keep only  $M \leftarrow M[\text{app\_id}, \text{app\_name}]$ 
7:  $R \leftarrow \text{LEFTJOIN}(R, M, \text{key}=\text{app\_id})$ 

8: Expand to dense (review, aspect) pairs
9: function BUILDPAIRS( $R, A$ )
10:    $P \leftarrow \emptyset$ 
11:   for each review row  $r \in R$  do
12:     for each aspect  $a \in A$  do
13:       Append record to  $P$ :
       {app_id= $r[\text{app\_id}]$ , app_name= $r[\text{app\_name}]$ ,
        store= $r[\text{store}]$ , review_id= $r[\text{review\_id}]$ ,
        text= $r[\text{text}]$ , aspect= $a$ }
14:     end for
15:   end for
16:   return DATAFRAME( $P$ )
17: end function
18:  $P \leftarrow \text{BUILDPAIRS}(R, A)$ 

19: Sanitize inputs (required for tokenisation)
20:  $P[\text{text}] \leftarrow$  fill missing with empty string and cast to string
21:  $P[\text{aspect}] \leftarrow$  fill missing with empty string and cast to string

22: Load model
23:  $T \leftarrow \text{AUTOTOKENIZER.FROM\_PRETRAINED}(\text{MODEL\_DIR})$ 
24:  $f_\theta \leftarrow \text{AUTOMODELFORSC.FROM\_PRETRAINED}(\text{MODEL\_DIR})$ 
25: Set  $f_\theta$  to evaluation mode:  $f_\theta.\text{EVAL}()$ 
26:  $\text{id2label} \leftarrow$  cast keys in  $f_\theta.\text{config.id2label}$  to integers

27: Convert pairs to HuggingFace Dataset
28:  $HF \leftarrow \text{DATASET.FROM\_PANDAS}(P)$ 

29: Tokenise with batched mapping
30: function TOKENIZEBATCH( $batch$ )
31:   return  $T(batch[\text{text}], batch[\text{aspect}])$ 
32: end function
33:  $HF \leftarrow HF.\text{MAP}(\text{TOKENIZEBATCH}, \text{batched} = \text{True})$ 
34:  $HF.\text{SET\_FORMAT}(\text{type} = \text{torch}, \text{columns} = \{\text{input\_ids}, \text{attn\_mask}\})$ 

```

---

---

**Algorithm 6** Batched Inference, Decoding, and Output for Government-App Review Pairs

---

**Require:** tokenised HF dataset  $HF$  with `input_ids`, `attn_mask`; data collator; model  $f_\theta$ ; evaluation batch size  $B$ ; label map `id2label`

```

1: Create DataLoader with dynamic padding
2:  $collator \leftarrow \text{DataCollatorWithPadding}(\text{tokenizer}=T,$ 
    $\text{return_tensors}="pt")$ 
3:  $loader \leftarrow \text{DataLoader}(HF, \text{batch\_size}=B, \text{shuffle}=\text{False},$ 
    $\text{collate\_fn}=collator)$ 

4: Run batched forward passes (no gradients)
5:  $AllLogits \leftarrow []$ 
6: with NO_GRAD do
7:   for each batch  $b$  in  $loader$  do
8:     Move tensors to device:  $b \leftarrow \{k : v.\text{TO}(\text{DEVICE}) \mid \forall (k, v) \in b\}$ 
9:      $Z \leftarrow f_\theta(b).\text{logits}$   $\triangleright Z \in R^{|b| \times 4}$ 
10:    Append  $Z.\text{CPU}()$  to  $AllLogits$ 
11:   end for
12:  $L \leftarrow \text{CONCAT}(AllLogits, \text{dim}=0)$   $\triangleright$  logits matrix

13: Compute probabilities (numerically stable softmax)
14:  $L \leftarrow \max(L, \text{axis}=1, \text{keepdims}=\text{True})$ 
15:  $E \leftarrow \exp(L - L)$ 
16:  $P \leftarrow E / \sum(E, \text{axis}=1, \text{keepdims}=\text{True})$   $\triangleright P$  is class-probability matrix

17: Decode predicted labels and confidences
18:  $pred\_id \leftarrow \arg\max(P, \text{axis}=1)$ 
19:  $pred\_label \leftarrow \text{id2label}[pred\_id]$ 
20:  $pred\_prob \leftarrow P[\text{row\_index}, pred\_id]$   $\triangleright$  probability of predicted class

21: Assemble output CSV
22: Attach columns to the original pair table:
    $pred\_label\_id \leftarrow pred\_id$ 
    $pred\_label\_str \leftarrow pred\_label$ 
    $prob \leftarrow pred\_prob$ 

23: return prediction table for all  $(review, aspect)$  pairs

```

---

### 3.4 Explainability Pipeline Overview

The previous sections describe how the fine-tuned DistilRoBERTa aspect-based sentiment analysis model is applied to newly collected reviews of North American federal, state, and provincial government mobile applications, producing aspect-level sentiment predictions for each (review, aspect) pair. Although these predictions provide a useful quantitative summary of user satisfaction across many functional facets of government



services, they are inherently difficult to interpret. Neither users, researchers or developers can easily determine which sections of the review text influence a given sentiment classification. Additionally, by making the model’s reasoning more transparent, the XAI outputs can be directly used by product teams and developers to locate genuine pain points in specific aspects of an application, consequently fostering trust in the model’s decisions and allowing them to move more quickly toward targeted design and engineering changes rather than first having to manually audit or reinterpret the underlying predictions.

These issues are addressed by introducing an explainable AI (XAI) layer that incorporates gradient-based attribution (Integrated Gradients Captum), large language model generated textual justifications (Gemini), and a set of quantitative explanation quality metrics. By highlighting important textual cues, producing comprehensible justifications, and methodically evaluating the behavior of the explanations, this XAI pipeline aims to close the gap between model internals and human interpretation.

The pipeline operates on the output of the inference stage (the predictions file with aspect-level labels) and consists of four main stages listed in the Table 7 below and will be described in further detail in the following sections.

**Table 7.** Overview of explainability pipeline with descriptions

| Explainability Pipeline Stage                | Brief Description                                                                                                                                                                                                                   |
|----------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Text Preprocessing for XAI                   | Reviews are processed with spaCy using a customized stopwords set that preserve cues that could link to any sentiment (for example, negation and intensifier words).                                                                |
| Integrated Gradients based token attribution | For each non-absent prediction, Captum’s Integrated Gradients (IG) is used to calculate token-level attributions over the Distil-RoBERTa input embeddings, resulting in positive and negative evidence for the predicted sentiment. |
| Gemini based textual justifications          | A stratified sample of (app, aspect, label) instances is passed to Google’s Gemini Flash 2.5 model, together with the IG derived evidence phrases, to generate short natural language justifications for the model’s decisions.     |
| XAI metrics                                  | A set of explanation quality metrics is computed by perturbing the text based on the IG scores and analyzing the behavior of both the classifier and the explanation text.                                                          |

**Text Preprocessing for XAI with spaCy.** For explanation generation, an additional text preprocessing layer is applied on top of the public sector mobile application reviews which specifically targets at controlling which tokens are allowed to appear as candidate “evidence” in IG-based explanations. The spaCy `en_core_web_sm` pipeline is utilized to obtain the default list English stopwords, and then this set is customized in two ways. First, the sentiment-based function words, particularly negation and intensity terms such as no, not, never, very and extremely are removed. Considering that

they are polarity shifters this makes them crucial in determining the sentiment of a review and should therefore remain available as candidate evidence tokens. Second, a comprehensive set of English pronouns is added to the stopword list, including subject, object, possessive, reflexive, demonstrative and relative forms. This suppresses low-content personal references such as I, you, we, and they, which rarely contribute any meaningful semantic meaning for aspect-level sentiment and would otherwise dilute the quality of the explanation phrases.

A helper function `is_informative(token)` then filters candidate fragments by discarding tokens that are empty, belong to customized stopword set, consist only of punctuation, or are single character tokens. In order to ensure the only linguistically significant fragments are returned as “support or “oppose” cues, this filter is used when extracting phrases from offset spans that the tokenizer returns. The complete sequence of preprocessing operations applied before attribution extraction is summarized in Table 8 below.

**Table 8.** spaCy based text preprocessed steps. Used to control which textual fragments are eligible as explanation evidence in the XAI pipeline.

| Step                                                     | Description                                                                                                                                                                                                                                                                                                     |
|----------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Load spaCy model                                         | Loads the <code>en_core_web_sm</code> pipeline and initializes <code>nlp.Defaults.stop_words</code> as the base English stopword set.                                                                                                                                                                           |
| Negation and intensifier handling                        | Remove negation and intensity terms from the stopword set so that they remain eligible as explanation tokens.                                                                                                                                                                                                   |
| Pronoun Augmentation                                     | Adds a comprehensive list of English pronouns to the stopword set to suppress low-content personal references.                                                                                                                                                                                                  |
| Token-level filtering <code>is_informative()</code>      | Defines a filter that discards tokens that are empty, are in the customized stopword set, consist solely of punctuation, or are single character tokens. Only the tokens passing this filter are allowed into explanations phrases.                                                                             |
| Span Cleaning <code>clean_span()</code>                  | Trims trailing, leading punctuation, and whitespace around extracted spans, producing readable fragments that can be reused in IG and Gemini explanations.                                                                                                                                                      |
| Phrase Constructions <code>phrases_from_offsets()</code> | Merges top-ranked tokens based on attribution scores into sequential and unbroken sequences of tokens (contiguous spans) using offset information. Applies <code>is_informative()</code> and returns a de-duplicated list of salient phrases, with a fallback to raw tokens if all candidates are filtered out. |

**Integrated Gradients for Token-Level Attribution.** A methodical approach to attributing a model prediction to its input features is provided Integrated Gradients (IG). Given a model  $F$  and an input embedding vector  $x$  with baseline input  $x'$ , the IG attribution for feature  $i$  is defined as the following (Walker et al., 2024).

$$IG_i(x) = (x_i - x'_i) \int_0^1 \frac{\partial F(x' + \alpha(x - x'))}{\partial x_i} d\alpha \quad (10)$$

In practice, the IG path integral is approximated numerically by evaluating the model over a finite sequence of interpolated inputs. In this implementation, a forward function is constructed that maps input embeddings and attention masks directly to the classifier’s logits. The baseline input  $x'$  is defined as the embedding of the pad token replicated across the full sequence. The attributions are then calculated using Captum’s IntegratedGradients with  $n = 16$  interpolation steps to approximate the integral.

For each batch of (review, aspect) pairs, the IG attributions are calculated for the predicted class, the attributions are aggregated over the embedding dimension to obtain a scalar attribution per token, and the top-scoring positive and negative tokens are extracted. These token-level attributions are then merged into “supporting” and “opposing” phrases using offset mapping and the spaCy-based filters described in the previous section. This process is summarized in Table 9 and formalized in Algorithms 7, 8 and 9 below.

**Table 9.** Integrated Gradients attribution pipeline. Used to extract token and phrase-level explanations for model predictions.

| Step                              | Description                                                                                                                                                                                                                                                                     |
|-----------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Batch Tokenization                | For each batch of review texts and aspects, the tokenizer is applied with truncation, padding, and <code>return_offsets_mapping()</code> to obtain input IDs, attention masks, and character offsets for all subword tokens.                                                    |
| Forward pass and predicted class  | Runs the DistilRoBERTa model in no-gradient mode to obtain logits, converts them to probabilities with softmax, and selects the predicted class identifier for each sequence.                                                                                                   |
| Baseline and path construction    | Extracts the DistilRoBERTa model’s input embedding layer and constructs a baseline sequence by replacing all token IDs with the pad token ID. The original and baseline sequences are then embedded forming the endpoints of the IG interpolation path.                         |
| Integrated Gradients computation  | Uses Captum’s IntegratedGradients with the embedding-based forward function to calculate attributions for the predicted class over $n = 16$ interpolation steps. This results in an attribution vector per token and per embedding dimension.                                   |
| Token-level attribution score     | Aggregates attributions over the embedding dimension to obtain a scalar attribution per token. For each example, the top-k most positive and negative tokens within the review text sequence are identified.                                                                    |
| Phrase extraction and filtering   | Merges adjacent or near-adjacent top-scoring tokens into character spans, converts them to text fragments, and filters them through <code>is_informative()</code> . The resulting positive and negative phrase lists are stored as ‘support’ and ‘oppose’ evidence per example. |
| Export of explanations and scores | Attaches support/oppose phrases and XAI labels to the input prediction dataframe. Stores the per-example token attribution vectors in a separate file for metric computation.                                                                                                   |

---

**Algorithm 7** Integrated Gradients XAI: Setup, Filtering, and Configuration
 

---

**Require:** predictions file `PREDS_CSV` with columns `text`, `aspect`, `pred_label_str`; fine-tuned model directory `MODEL_DIR`; output paths `OUTPUT_CSV`, `TOKEN_SCORES_CSV`;

1: **Configuration**

2:  $\beta \leftarrow \{\text{BATCH}=4, \text{IG\_STEPS}=16, \text{TOPK}=6, \text{USE\_ASPECT\_PAIR}=\text{True}\}$

3: **Load tokenizer and saved model checkpoint**

4:  $T \leftarrow \text{AUTOTOKENIZER.FROM\_PRETRAINED}(\text{MODEL\_DIR}, \text{use\_fast}=\text{True})$

5:  $f_\theta \leftarrow \text{AUTOMODELFORSC.FROM\_PRETRAINED}(\text{MODEL\_DIR})$

6: Move  $f_\theta$  to device and set evaluation mode:  $f_\theta.\text{EVAL}()$

7: **Load predictions and filter evaluation rows**

8:  $D \leftarrow \text{READ\_CSV}(\text{PREDS\_CSV})$

9: Filter:  $D \leftarrow D[D[\text{pred\_label\_str}] \neq \text{"absent"}]$

10: Reset row indices for stable batching:  $D \leftarrow D.\text{RESET\_INDEX}(\text{drop}=\text{True})$

11: **Extract input lists used for batching**

12:  $\text{texts} \leftarrow D[\text{text}].\text{ASTYPE}(\text{STR}).\text{TOLIST}()$

13:  $\text{aspects} \leftarrow D[\text{aspect}].\text{ASTYPE}(\text{STR}).\text{TOLIST}()$

14: **Initialize Integrated Gradients and embedding layer**

15:  $E(\cdot) \leftarrow f_\theta.\text{GET\_INPUT\_EMBEDDINGS}()$   $\triangleright$  token embedding module

16: Define forward function:

$g(\text{emb}, \text{attn}) \leftarrow f_\theta(\text{inputs\_embeds} = \text{emb}, \text{attn\_mask} = \text{attn}).\text{logits}$

17:  $IG \leftarrow \text{INTEGRATEDGRADIENTS}(g)$

18: **Initialize outputs (per-row)**

19: Initialize lists: `support_list`, `oppose_list`, `pred_label_xai`, `pred_probs_xai`

20: Initialize lists: `token_row_idx`, `token_scores`

---

---

**Algorithm 8** Integrated Gradients XAI: Batched tokenization, Target Selection, and Attribution

---

**Require:** tokenizer  $T$ ; model  $f_\theta$ ; embedding layer  $E$ ; IG object  $IG$ ; lists **texts**, **aspects**; configuration  $\beta$

```

1: function TOKENIZEBATCH( $t\_list$ ,  $a\_list$ )
2:    $enc \leftarrow T(t\_list, \text{text\_pair} = a\_list, \text{max\_length}=128,$ 
      $\text{return\_tensors}="pt", \text{return\_offsets\_mapping}=\text{True})$ 
3:    $offsets \leftarrow enc[\text{offset\_mapping}]$  and remove from  $enc$ 
4:    $encodings \leftarrow enc.\text{encodings}$ 
5:   return ( $enc$ ,  $offsets$ ,  $encodings$ )
6: end function

7: function IGBATCH( $t\_list$ ,  $a\_list$ )
8:   ( $enc, offsets, encodings$ )  $\leftarrow$  TOKENIZEBATCH( $t\_list$ ,  $a\_list$ )
9:    $ids \leftarrow enc[\text{input\_ids}].\text{TO}(\text{DEVICE})$ 
10:   $attn \leftarrow enc[\text{attn\_mask}].\text{TO}(\text{DEVICE})$ 

11:  Forward pass to determine IG targets (predicted class per example)
12:  with NO_GRAD do
     $logits \leftarrow f_\theta(\text{input\_ids} = ids, \text{attn\_mask} = attn).\text{logits}$ 
     $probs \leftarrow \text{SOFTMAX}(logits, \text{dim} = -1)$ 
     $preds \leftarrow \arg\max(probs, \text{dim} = -1)$ 

13:  Construct embeddings and baseline embeddings
14:   $embeds \leftarrow E(ids)$   $\triangleright \text{shape} : \text{Batch Size} \times \text{Seq Length} \times \text{dim}(E)$ 
15:   $pad\_id \leftarrow T.\text{pad\_token\_id}$   $\triangleright$  fallback:  $\text{eos\_token\_id}$  else 0 if missing
16:  if  $pad\_id$  is None then
17:     $pad\_id \leftarrow T.\text{eos\_token\_id}$  if defined else 0
18:  end if
19:   $base\_ids \leftarrow$  tensor filled with  $pad\_id$ , same shape as  $ids$ 
20:   $base\_embeds \leftarrow E(base\_ids)$ 

21:  Apply Integrated Gradients w.r.t. predicted class
22:   $atts \leftarrow IG.ATTRIBUTE($ 
      $\text{inputs}=embeds, \text{baselines}=base\_embeds,$ 
      $\text{additional\_forward\_args} = (attn, ),$ 
      $\text{target} = preds, \text{n\_steps} = 16)$ 
23:  Reduce to per-token attribution scores
24:   $token\_scores \leftarrow \sum_{j=1}^d atts[:, :, j]$   $\triangleright$  sum over embedding dim
25:  return ( $token\_scores$ ,  $probs$ ,  $preds$ ,  $offsets$ ,  $encodings$ )
26: end function

```

---

---

**Algorithm 9** Integrated Gradients XAI: Evidence Phrase Extraction and Export

---

**Require:** lists `texts`, `aspects`; function `IGBATCH`; `TOPK=6`; `ID2LABEL`; output lists (`support_list`, `oppose_list`, `pred_label_xai`, `pred_probs_xai`, `token_row_idx`, `token_scores`)

```

1: function EXTRACTTOPPHRASES(text, seq_ids, offsets, scores, k)
2:   Restrict to text tokens only:
3:   keep indices  $J = \{j : seq\_ids[j] = 0 \wedge offsets[j] \neq (0, 0)\}$ 
4:   Let  $s[j]$  be attribution score for token  $j$  (from scores)
5:   Select top- $k$  tokens by descending  $s[j]$  as supporting set  $J^+$ 
6:   Select top- $k$  tokens by ascending  $s[j]$  as opposing set  $J^-$ 
7:   Merge adjacent offsets inside  $J^+$  and  $J^-$  into spans ( $start, end$ )
8:   Convert each span to substring  $text[start : end]$ , apply cleaning, de-
     duplication, and IS_STOPWORD filtering
9:   return (support_phrases, oppose_phrases)
10: end function

11: Iterate in batches over all non-absent examples
12: for  $i_0 = 0$  to  $|texts|$  step 4 do
13:    $b\_texts \leftarrow texts[i_0 : i_0 + 4]$ 
14:    $b\_aspects \leftarrow aspects[i_0 : i_0 + 4]$ 
15:   ( $S$ ,  $P$ ,  $\hat{y}$ ,  $Off$ ,  $Enc$ )  $\leftarrow$  IGBATCH( $b\_texts$ ,  $b\_aspects$ )
16:                                      $\triangleright S$  is token-score matrix for the batch
17:   for  $b_i = 0$  to  $|b\_texts| - 1$  do
18:      $seq\_ids \leftarrow Enc[b_i].sequence\_ids$   $\triangleright 0=text, 1=aspect, None=special$ 
19:     ( $pos, neg$ )  $\leftarrow$  EXTRACTTOPPHRASES( $b\_texts[b_i]$ ,  $seq\_ids$ ,  $Off[b_i]$ ,  $S[b_i]$ , 6)
20:      $\hat{c} \leftarrow \text{int}(\hat{y}[b_i])$ 
21:     Append predicted label:  $pred\_label\_xai \leftarrow ID2LABEL[\hat{c}]$ 
22:     Append full probability distribution:  $pred\_probs\_xai \leftarrow P[b_i]$ 
23:     Append evidence phrases:
24:        $support\_list \leftarrow "$ ; ".join( $pos$ )
25:        $oppose\_list \leftarrow "$ ; ".join( $neg$ )
26:     Append token-level scores for later XAI metrics:
27:        $token\_row\_idx \leftarrow (i_0 + b_i)$ 
28:        $token\_scores \leftarrow S[b_i]$ 
29:   end for
30: end for

31: Merge and export
32: Attach support, oppose, pred_label_xai, pred_probs_xai to the
    dataframe
33: Save dataframe to OUTPUT_CSV
34: Save token-scores (row_idx, token_scores) to TOKEN_SCORES_CSV

35: return per-example evidence phrases and token-level attributions for all
    non-absent predictions

```

---

**Gemini-Based Natural Language Justifications.** Google’s Gemini Flash 2.5 model is used to complement the gradient-based attribution by generating short human readable textual justifications for a stratified subset of model predictions. The key idea is to condition the LLM on the original review text, the predicted sentiment label, and the IG derived support/oppose phrases, then ask it to explain the decision in 2-4 sentences targeted at a product team.

Controlling the cost and redundancy became crucial due to token and API request restrictions on Gemini’s free version. The explanation dataset is first grouped by app\_id, aspect, pred\_label\_str then sampled by a fixed number of examples per group (MAX\_PER\_GROUP = 10) and a global maximum (MAX\_ROWS\_GLOBAL = 100). For each sampled example, a prompt is constructed that includes the review text, the aspect, the predicted label, and the evidence phrases. Gemini is instructed to base its short justification only on the provided evidence, use the opposing cues if present to discuss nuances or mixed signals, and to avoid hallucinating new facts. The resulting justifications are stored in a gemini\_justification column for qualitative inspection. This process is detailed in Algorithm 10 and the exact prompt is provided in Appendix A.

---

**Algorithm 10** Gemini-Based Natural-Language Justifications for Stratified Model Predictions

---

**Require:** IG Explanation dataframe  $D$  containing columns: `text`, `app_id`, `aspect`, `pred_label_str`, `support_topk`, `oppose_topk`; Gemini API key `GEMINI_API_KEY`; Gemini model  $m = \text{gemini-2.5-flash}$ ; sampling limits `MAX_PER_GROUP` = 10, `MAX_ROWS_GLOBAL` = 100; rate limit delay  $\Delta t = 10$  seconds; random seed  $s = 58$ .

```

1: Initialize Gemini client
2: Configure SDK with GEMINI_API_KEY
3: Instantiate generative model  $G \leftarrow \text{GENERATIVE\_MODEL}(m)$ 

4: Index rows for stable merging
5: Reset DataFrame index and store original index as row_idx

6: Stratified sampling to control cost and redundancy
7: Define grouping keys  $C = (\text{app\_id}, \text{aspect}, \text{pred\_label\_str})$ 
8: Group  $D$  by  $C$  into  $\{G_k\}$ 
9: Initialise empty list  $S$ 
10: for each group  $G_k$  do
11:   Sample  $\min(|G_k|, \text{MAX\_PER\_GROUP})$  rows using seed  $s$ 
12:   Append sampled rows to  $S$ 
13: end for
14: if  $|S| > \text{MAX\_ROWS\_GLOBAL}$  then
15:   Subsample  $S$  to MAX_ROWS_GLOBAL using seed  $s$ 
16: end if
17: Reset index of  $S$ 

18: Generate justifications
19: for each row  $r \in S$  do
20:   Build prompt  $p$  using BUILD\_PROMPT(r) ▷ Appendix A
21:    $j \leftarrow G.\text{GENERATE\_CONTENT}(p)$ 
22:   Store generated text as justification
23:   SLEEP( $\Delta t$ ) ▷ API rate limiting
24: end for

25: Write outputs
26: Insert justifications into  $D$  at indices row_idx
27: Save sampled justification dataframe

28: return Stratified subset with Gemini-generated justifications

```

---



**Explainability Metrics.** To evaluate the quality, stability, and interpretability of the explanations produced by Integrated Gradients and the Gemini rationales, a set of quantitative explainability metrics are calculated. The selection is guided by the `xai_evals` framework proposed by Seth et al. (2025), which defies model-agnostic criteria for assessing the behavioral reliability of feature-attribution methods. Although originally formulated for tabular settings, several of these metrics naturally extend to the token-level attributions generated in aspect-based sentiment analysis. In this work, the definitions are adapted to the text domain while preserving their conceptual meaning. The metrics fall into two groups, firstly, the adapted `xai_evals` metrics which include comprehensiveness, sufficiency, sparsity, and local sensitivity following the mathematical structure of the original paper. Secondly, the NLP-specific extensions include probability-based monotonicity and polarity alignment, but are not defined by Seth et al. (2025). The paragraphs that follow provide a detailed description of the two groups of metrics adopted for evaluating explanations along with a formalized Algorithm 11.

Comprehensiveness measures how much the model’s confidence in its predicted label decreases when the most important tokens (according to their attribution scores) are removed from the input. Following Seth et al. (2025), this metric is interpreted as a measure of the sufficiency of the identified evidence where the truly important features should, when removed, cause a significant drop in prediction strength. Let  $x$  denote the original review, and  $x_{\text{remove-top-}K}$  the text with the top- $K$  tokens (by attribution magnitude) removed. Given that  $f(x)$  denotes the model’s predicted probability of the chosen class, the per-example comprehensiveness metric is defined as,

$$\text{Comprehensiveness}(x) = f(x) - f(x_{\text{remove-top-}K}) \quad (11)$$

and the overall metric is the mean of  $\text{comprehensiveness}(x)$  across examples. Larger values from this metric indicate that the highlighted tokens are truly important for the prediction.

Sufficiency assesses whether the top- $K$  highest attribution tokens alone are adequate to preserve the predicted sentiment. A minimal text  $x_{\text{keep-top-}K}$  containing only these salient tokens is constructed and the metric is defined as,

$$\text{Sufficiency}(x) = f(x) - f(x_{\text{keep-top-}K}) \quad (12)$$

where low values indicate that these tokens alone are enough to support the prediction, implying concentrated and well-localized explanations.

Sparsity quantifies how concentrated the attribution mass is across the token sequence. Computing token-level attribution scores  $a_i$ , the sparsity metric is defined as,

$$\text{Sparsity}(x) = \frac{1}{n} \sum_{i=1}^n (|a_i| = 0) \quad (13)$$

measuring the fraction of zero-importance features. Higher values from this calculation correspond to more compact explanations.

Local sensitivity evaluates the robustness of attribution scores under small perturbations to the explanation itself. Following the definition in xai\_evals, a small Gaussian perturbation  $\epsilon \sim \mathcal{N}(0, \sigma^2)$  is applied to the attributions producing the local sensitivity,

$$\text{Sensitivity}(x) = \frac{1}{n} \sum_{i=1}^n |a_i - a'_i| \quad ; \quad a'_i = a_i + \epsilon_i \quad (14)$$

where lower values indicate that attribution are relatively stable under minor noise.

For monotonicity, the definition in Seth et al.(2025) is formulated through sign consistency, which evaluates whether attribution scores maintain consistent directional behavior when perturbations are applied. However, because the DistilRoBERTa model outputs probabilities over discrete sentiment classes rather than continuous regression values, the sign consistency is replaced with a probability-based monotonicity measure that is more appropriate for classification settings. The underlying mathematical logic remains the same where progressively adding higher-attribution tokens should not reduce the model’s confidence in its original prediction. Formally, let  $i_1, i_2, \dots, i_K$  denote the indices of the top-K tokens ranked by absolute Integrated Gradients attribution and for each prefix set  $S_k = \{i_1, i_2, \dots, i_k\}$ . The model’s predicted probability denoted  $p_k$ , is calculated for the original label  $y$  using only the tokens in  $S_k$ . The monotonicity score is then defined as the proportion of probability increments that are non-decreasing, and mathematically expressed as,

$$\text{Monotonicity} = \frac{1}{K-1} \sum_{k=1}^{K-1} 1[p_{k+1} \geq p_k]. \quad (15)$$

It offers a natural and interpretable adaption of xai\_eval’s monotonicity criterion to aspect-based sentiment analysis classification, where probability behavior provides a clearer signal of explanation faithfulness than raw attribution sign consistency. Values closer to 1 indicate that the highest-attribution tokens truly contribute positively and cumulatively to the model’s prediction.

Polarity alignment is a custom metric used to assess whether extracted phrases align with human-interpretable sentiment polarity, the VADER sentiment analyzer is applied to both support and oppose phrase sets. For extracted support phrases  $S$ , the polarity alignment metric is defined as,

$$\text{Polarity}(S) = \text{VADER}(S)_{\text{compound}} \quad (16)$$

and analogously for oppose phrases  $O$ . Higher positive polarity for support cues and negative polarity for opposing cues indicate semantically coherent explanations.

---

**Algorithm 11** XAI Metric Computation from IG Token Scores (Exact Implementation)

---

**Require:** Predictions file `review_preds_with_xai.csv` containing at least: `text`, `aspect`, `pred_label_str`, `support`, `oppose`; Token score file `review_token_scores_ig.csv` containing: `row_idx`, `token_scores` (stringified list); model  $f_\theta$  and tokenizer  $T$  loaded from `MODEL_DIR`; VADER sentiment analyser `SENTIMENTINTENSITYANALYZER`; label list  $\mathcal{L} = [\text{negative}, \text{neutral}, \text{positive}, \text{absent}]$ .

```

1: Set constants (exact values used)
2:  $K \leftarrow 5$  ▷ top- $K$  tokens kept/removed for masking metrics
3:  $L_{\max} \leftarrow 256$  ▷ max sequence length in PREDICT_PROBABILITIES
4:  $\epsilon \leftarrow 10^{-4}$  ▷ Gaussian noise std for local sensitivity
5:  $K_{\text{mono}} \leftarrow 6$  ▷ monotonicity evaluates  $k = 1, \dots, \min(5, |I|)$ 
6:  $\mathcal{L} \leftarrow [\text{negative}, \text{neutral}, \text{positive}, \text{absent}]$ 
7:  $\text{LABEL2ID} \leftarrow \{\ell \mapsto i \mid \ell = \mathcal{L}[i]\}$ 

8: Load and merge evaluation table
9: Load  $df_{\text{preds}} \leftarrow \text{READ\_CSV}(\text{review\_preds\_with\_xai.csv})$ 
10: Reset index of  $df_{\text{preds}}$  and store old index as row_idx
11: Load  $df_{\text{scores}} \leftarrow \text{READ\_CSV}(\text{review\_token\_scores\_ig.csv})$ 
12: Parse token_scores strings using LITERAL\_EVAL; on failure return empty list
13:  $df_{\text{eval}} \leftarrow \text{MERGE}(df_{\text{preds}}, df_{\text{scores}}, \text{on}=\text{row\_idx}, \text{how}=\text{inner})$ 

14: Initialise VADER and evaluation device
15:  $\text{analyzer} \leftarrow \text{SENTIMENTINTENSITYANALYZER}()$ 
16: Move  $f_\theta$  to device and set  $f_\theta.\text{EVAL}()$ 

17: Define helper: polarity(text)
18: function POLARITY( $x$ )
19:   if  $x$  is not a string OR  $x$  is empty then
20:     return 0.0
21:   end if
22:   return VADERCOMPOUND( $x$ ) ▷ = analyzer.polarity_scores(x)["compound"]
23: end function

24: Define helper: predict_probabilities(text, aspect)
25: function PREDICT_PROBABILITIES( $t, a$ )
26:   if  $t$  is not a string then
27:     coerce:  $t \leftarrow ""$  if NaN else STR( $t$ )
28:   end if
29:   if  $a$  is not a non-empty string then
30:      $a \leftarrow \text{None}$  ▷ disable text_pair if missing
31:   end if
32:    $\text{enc} \leftarrow T(t, \text{text\_pair} = a,$ 
33:     truncation=True, padding=True, max\_length= $L_{\max}$ ,
34:     return\_tensors="pt")
35:   Move  $\text{enc}$  to device
36:   with no gradient:
37:      $\text{logits} \leftarrow f_\theta(\text{enc}).\text{logits}$ 
38:      $p \leftarrow \text{SOFTMAX}(\text{logits})[0]$  moved to CPU numpy
39:   return  $p$  ▷ length  $|\mathcal{L}|$  probability vector
40: end function

```

---

---

```

1: Define helper:   ig_mask_text(text, scores, keep_top_k, re-
   move_top_k)
2: function IG_MASK_TEXT( $x$ ,  $\mathbf{a}$ ,  $K_{\text{keep}}$ ,  $K_{\text{remove}}$ )
3:   if  $x$  is not a string then
4:      $x \leftarrow ""$  if NaN else STR( $x$ )
5:   end if
6:    $\text{tokens} \leftarrow$  whitespace-split of  $x$ 
7:    $\mathbf{s} \leftarrow$  attribution scores as NumPy array
8:   if  $|\text{tokens}| = 0$  OR  $|\mathbf{s}| = 0$  then
9:     return  $x$ 
10:  end if
11:   $L \leftarrow \min(|\text{tokens}|, |\mathbf{s}|)$ 
12:  Truncate  $\text{tokens} \leftarrow \text{tokens}[1:L]$ 
13:  Truncate  $\mathbf{s} \leftarrow \mathbf{s}[1:L]$ 

14:  if  $K_{\text{keep}}$  is not None then
15:     $K \leftarrow \min(K_{\text{keep}}, L)$ 
16:     $I \leftarrow$  indices of top- $K$  values in  $\mathbf{s}$ 
17:     $\text{tokens}' \leftarrow \{\text{tokens}[i] : i \in I\}$ 
18:    return tokens joined by spaces
19:  end if

20:  if  $K_{\text{remove}}$  is not None then
21:     $K \leftarrow \min(K_{\text{remove}}, L)$ 
22:     $I \leftarrow$  indices of top- $K$  values in  $\mathbf{s}$ 
23:     $\text{tokens}' \leftarrow \{\text{tokens}[i] : i \notin I\}$ 
24:    return tokens joined by spaces
25:  end if
26:  return  $x$ 
27: end function

```

---

---

```

1: Initialise per-row metric accumulators
2: Create empty lists: comprehensiveness, sufficiency, sparsity,
   local_sensitivity, monotonicity, support_polarity, oppose_polarity

3: Compute metrics for each example  $r$ 
4: for each row  $r$  in  $df_{eval}$  do
5:    $t \leftarrow r.text$ ;  $a \leftarrow r.aspect$ 
6:    $\ell \leftarrow r.pred\_label\_str$ ;  $y \leftarrow LABEL2ID[\ell]$ 
7:    $s \leftarrow$  numpy array from  $r.token\_scores$ 
8:    $u \leftarrow |s|$   $\triangleright$  absolute attributions used everywhere

9:   Original predicted-class probability
10:   $p \leftarrow PREDICT\_PROBABILITIES(t, a)$ 
11:   $p_{orig} \leftarrow p[y]$ 

12:   Comprehensiveness (remove top- $K$  tokens)
13:   $t_{rem} \leftarrow IG\_MASK\_TEXT(t, u, None, K)$ 
14:   $p_{rem} \leftarrow PREDICT\_PROBABILITIES(t_{rem}, a)$ 
15:  Append  $p_{orig} - p_{rem}[y]$  to comprehensiveness

16:   Sufficiency (keep only top- $K$  tokens)
17:   $t_{keep} \leftarrow IG\_MASK\_TEXT(t, u, K, None)$ 
18:   $p_{keep} \leftarrow PREDICT\_PROBABILITIES(t_{keep}, a)$ 
19:  Append  $p_{orig} - p_{keep}[y]$  to sufficiency

20:   Sparsity (exact definition used in code)
21:   $sp \leftarrow MEAN(u = 0)$ 
22:  Append  $sp$  to sparsity

23:   Local sensitivity (Gaussian perturbation of attribution vector)
24:  Draw  $\eta \sim \mathcal{N}(0, \epsilon^2)$  with length  $|u|$ 
25:   $u' \leftarrow u + \eta$ 
26:   $ls \leftarrow MEAN(|u' - u|)$ 
27:  Append  $ls$  to local_sensitivity

28:   Monotonicity (probability non-decrease as top- $k$  grows)
29:   $I \leftarrow$  indices of  $u$  sorted descending
30:  Initialise empty list  $P$ 
31:  for  $k = 1$  to  $\min(K_{mono} - 1, |I| - 1)$  do
32:     $t_{(k)} \leftarrow IG\_MASK\_TEXT(t, u, k, None)$ 
33:     $p^{(k)} \leftarrow PREDICT\_PROBABILITIES(t_{(k)}, a)$ 
34:    Append  $p^{(k)}[y]$  to  $P$ 
35:  end for
36:  if  $|P| > 1$  then
37:     $mono \leftarrow MEAN(\Delta P \geq 0)$   $\triangleright \Delta P =$  consecutive differences
38:  else
39:     $mono \leftarrow NaN$ 
40:  end if
41:  Append  $mono$  to monotonicity

42:   Polarity alignment (VADER compound on cue strings)
43:  Append  $POLARITY(r.support)$  to support_polarity
44:  Append  $POLARITY(r.oppose)$  to oppose_polarity
45: end for

```

---

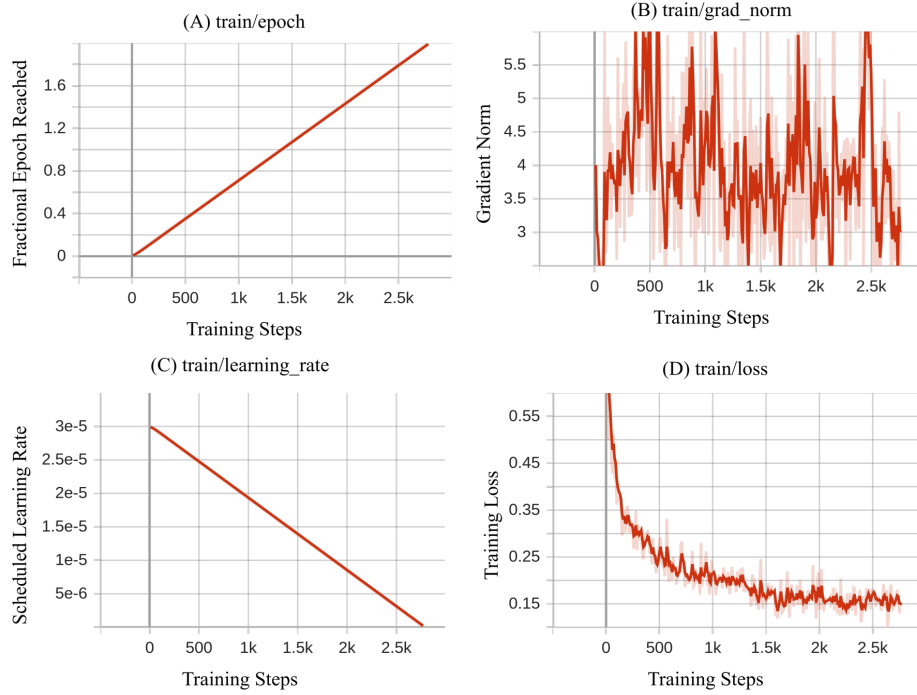
- 
- 1: **Attach metrics and compute mean summary**
  - 2: Add all metric lists as new columns in  $df_{\text{eval}}$
  - 3: Compute mean over: `comprehensiveness`, `sufficiency`, `sparsity`,  
`local_sensitivity`, `monotonicity`, `support_polarity`, `oppose_polarity`
  - 4: **return**  $df_{\text{eval}}$  with per-row metrics and global mean summary vector
-

## 4 Results

This section presents the empirical findings of the proposed framework, covering model training dynamics, quantitative performance on the FABSA test set, predictions on public sector mobile app reviews, explainability outputs, and diagnostic assessments of explanation quality. Together, these results demonstrate both the predictive capability of the fine-tuned DistilRoBERTa model and the interpretability presented by the layered XAI approach.

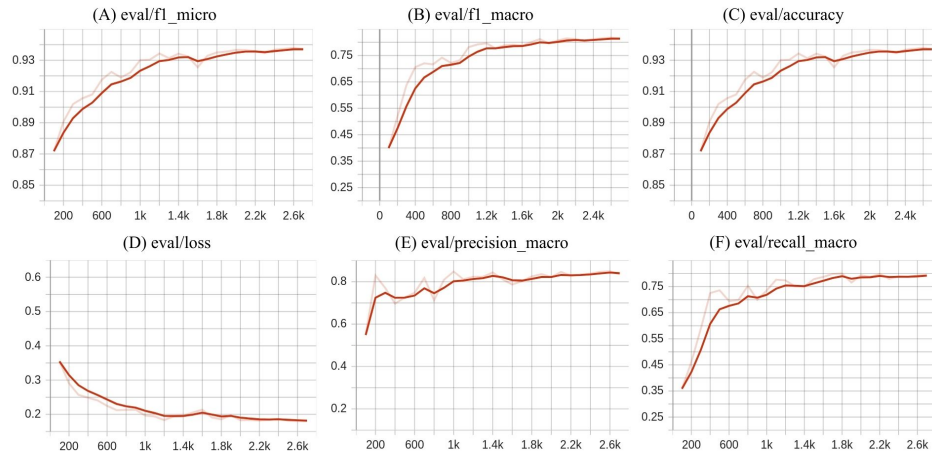
### 4.1 Training Dynamics

To characterize the optimization behavior of the fine-tuned DistilRoBERTa ABSA model, a full suite of HuggingFace Trainer logs were tracked and visualized using Tensorboard over 2.7k update steps ( $\approx 2$  epochs over the expanded sentence-aspect training set). Figures 6 and 7 illustrate the evolution of training signals and validation performance. These visualizations provide insight into convergence, stability, and generalization patterns across the fine-tuning process.



**Fig. 6.** Training-side optimization signals recorded. Panel (A) plots the epoch progression as a function of training steps and since HuggingFace logs epochs cumulatively, this increases linearly as batches are processed. Panel (B) displays gradient-norm fluctuations reflecting the magnitude of parameter updates at each optimization step, which stabilize as training proceeds. Panel (C) illustrates the scheduled linear decay of the learning rate from  $3 \times 10^{-5}$  to near 0. Panel (D) reports the training loss curve which decreases sharply early on and converges smoothly, indicating stable and effective optimization.

The training-side metrics in Figure 6 reflects a stable and well-regulated optimization process. Gradient norms (Panel B) remain within a normal range throughout training, suggesting that the gradient accumulation and the learning-rate schedule prevent instability or exploding updates. The linear learning-rate decay (Panel C) aligns with the smooth reduction in training loss (Panel D), implying that the model transitions effectively from rapid early learning to fine-grained adjustment. Overall, these trends reflect a training run that converges cleanly without signs of overfitting or optimization noise.



**Fig. 7.** Validation-side performance metrics on the development set. Panel (A) shows a steady increase in micro-F1 where each evaluation step corresponds to one evaluation interval during training. Panel (B) tracks macro-F1, which rises substantially as minority aspect-sentiment labels improve. Panel (C) displays accuracy, following the same upward trend. Panel (D) shows the validation loss decreasing without the signs of overfitting. Panels (E) and (F) plot macro-precision and macro-recall, respectively, both of which improve consistently over training, highlighting stronger performance on low-frequency classes.

Validation metrics in Figure 7 reinforce the view of reliable generalization. Micro-f1 and accuracy (Panels A and C) both show consistent improvements, indicating the model captures the dominant aspect patterns effectively. More importantly, macro-F1, macro-precision, and macro-recall (Panels B, E, F) rise substantially over training. This is particularly meaningful in the context of utilizing the FABSA dataset for training,



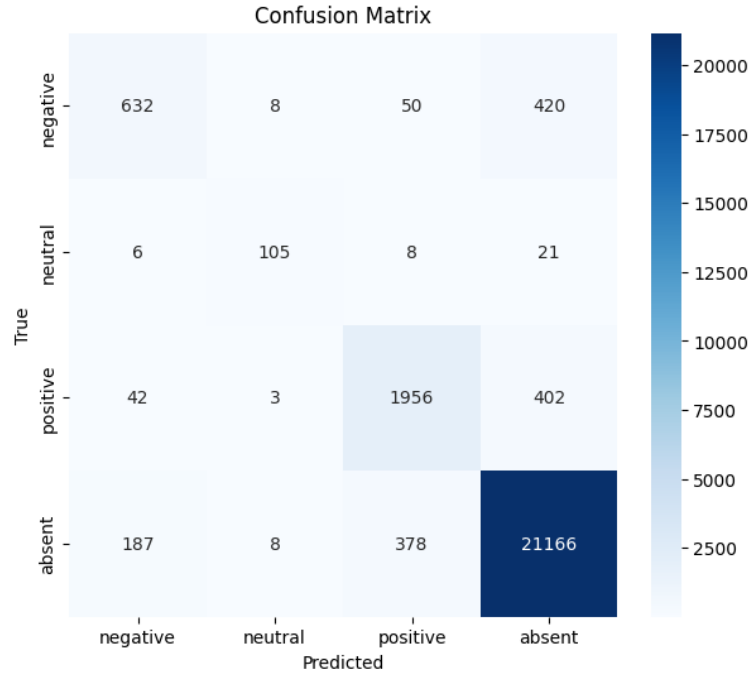
where many aspect-sentiment labels are infrequent. The improved macro-level performance demonstrates that the model performance becomes progressively better at handling rare classes rather than over-focusing on majority ones. The validation loss curve (Panel D) tracks the training loss closely, suggesting minimal overfitting and affirming that the training configuration (especially the gradient accumulation and schedules learning-rate decay) supports strong generalization. Everything considered, these results indicate that the fine-tuned DistilRoBERTa model is well suited for deployment on real-world review data where class imbalance and subtle cues are common.

## 4.2 Model Evaluation on FABSA Test Set

The fine-tuned DistilRoBERTa model was evaluated on the previously unseen `test_pairs` split following the aspect-sentiment formulation described in Section 3. Table 10 reports the micro and macro-averaged performance metrics computed using the HuggingFace evaluation utilities, along with Cohen’s Kappa to quantify the agreement beyond chance. Additionally, a confusion matrix is used to provide another perspective on the distribution of predictions across sentiment categories and highlights where the model makes systematic confusions.

**Table 10.** Overall Performance of the fine-tuned DistilRoBERTa model on the FABSA test set.

| Metric                     | Score  |
|----------------------------|--------|
| Accuracy                   | 0.9396 |
| F1 (Micro)                 | 0.9396 |
| F1 (Macro)                 | 0.8046 |
| Precision (Macro)          | 0.8388 |
| Recall (Macro)             | 0.7767 |
| Cohen’s Kappa ( $\kappa$ ) | 0.7561 |



**Fig. 8.** Confusion matrix showing predicted vs. true labels for negative, neutral, positive, and absent classes.

The overall performance is strong across all metrics. The high micro-F1 and accuracy (0.9396) indicate that the model reliably predicts sentiment across the majority of aspect-review pairs. Macro-F1 (0.8046) and macro-precision (0.8388), demonstrate solid performance even on minority sentiment classes, while macro-recall (0.7767) suggests the model captures a broad proportion of true positive instances across imbalanced labels. Cohen’s Kappa of 0.7561 indicates substantial agreement beyond chance, reinforcing the model’s reliability for downstream applications.

The confusion matrix in Figure 8, reveals clear strengths. Positive and absent classes show strong separability with high diagonal counts. The main sources of confusion occur between negative and positive labels, which often share overlapping lexical cues in user-generated text. These confusions underline the importance of deploying explainability techniques, as developers and analysts would require visibility into which specific textual cues drive borderline or incorrect predictions.

Collectively, these findings confirm that the fine-tuned model generalizes well and is suitable for large scale analysis of real-world reviews, while also motivating the integration of XAI methods to support error inspection and trust building.

### 4.3 Predictions on Real-World Government App Reviews

To evaluate the practical utility of the model, the fine-tuned DistilRoBERTa model was applied to a collection of user reviews from North American federal, state, and provincial government service applications as discussed in Section 3. These reviews span diverse domains including immigration processing, weather services, health records, public safety, and accessibility tools, providing a realistic collection for analyzing sentiment across digital public sector services. A stratified sample of predictions is presented in Table 11, illustrating the model’s outputs across multiple apps, stores, aspects and sentiment polarities.

**Table 11.** A stratified sample of aspect-level sentiment predictions for government mobile app reviews. Each row shows the app source, the extracted aspect, the model’s sentiment label, and the associated confidence, alongside a lightly truncated textual excerpt

| App Name         | Store     | Aspect       | Sentiment | Prob. | Excerpt                                                                  |
|------------------|-----------|--------------|-----------|-------|--------------------------------------------------------------------------|
| BARD             | Google    | attitude-of- | negative  | 0.85  | “...and can never reach anyone to help...”                               |
| Mobile           | Play      | staff        |           |       |                                                                          |
| WeatherCAN       | App Store | competitor   | positive  | 0.98  | “.....way more accurate that other weather apps”                         |
| NIOSH SLM        | App Store | ease-of-use  | positive  | 0.91  | “...The interface is clean and easy to navigate.....”                    |
| MyHealth         | Google    | account-ac-  | positive  | 0.78  | “...Nice to have access to all MY health records...”                     |
| Records          | Play      | cess         |           |       |                                                                          |
| Pneumo Recs      | App       | ease-of-use  | negative  | 0.58  | “...fix add passport using camera. Is not taking my passport...”         |
| VaxAdvisor       | Store     |              |           |       |                                                                          |
| WeatherCAN       | Google    | email        | absent    | 0.99  | “....app is good on my phone, but it isn't designed for a tablet....”    |
| Mobile Pass-     | Google    | competitor   | negative  | 0.59  | “...with and without this app are all in the same line for clearance...” |
| port Control     | Play      |              |           |       |                                                                          |
| Alternative      | Google    | general-sat- | negative  | 0.50  | “Doesn't work in Canada”                                                 |
| Fueling Stations | Play      | isfaction    |           |       |                                                                          |
| Job Bank -       | App Store | ease-of-use  | positive  | 0.91  | “This app is a lifesaver - so easy to use....”                           |
| Job Search       |           |              |           |       |                                                                          |
| BC Services      | Google    | price-       | absent    | 0.99  | “Helped me get onto my CRA account.”                                     |
| Card             | Play      | value-for-   |           |       |                                                                          |
|                  |           | money        |           |       |                                                                          |
| FEMA             | Google    | ease-of-use  | positive  | 0.87  | “Excellent easy to navigate...”                                          |
|                  | Play      |              |           |       |                                                                          |

Across the sampled predictions, the model demonstrates strong validity where negative sentiment clusters around usability breakdowns (“Doesn’t work”), lack of support (“never reach anyone for help”), and unmet expectations regarding functionality (“fix

add passport”). Positive predictions align with clear praise for design quality, reliability, and navigability all of which are attributes central to user trust in public-service applications.

A notable strength of the model, highlighted by the sample in Table 11, is its ability to assign sentiment at the aspect-level rather than the review level, enabling a more fine-grained interpretation of user feedback across government applications. For example, the WeatherCAN app receives positive sentiment on the competitor aspect (“way more accurate than other weather apps”) but is simultaneously assigned absent sentiment for aspects like email, correctly identifying that the review never touches on communication-related functionality. Similarly, apps such as BC Services Card receive absent sentiment for aspects they do not address, illustrating the model’s capacity to avoid hallucinating sentiment when the textual evidence does not support it. The Mobile Passport Control app also demonstrates clear aspect separation. The model assigns negative sentiment for competitor issues (“...same line for clearance..”), distinguishing process-level dissatisfaction from broader usability complaints.

This level of differentiation is particularly important in public-sector service delivery, where user feedback often mixes procedural complaints, staff interactions, technical errors, and usability frustrations within a single review. By disaggregating sentiment across aspects such as ease-of-use, account-access, general-satisfactions, and more, teams can pinpoint specific operational failure, for example, login workflows, navigation issues, or comparative underperformance without combining them with unrelated sentiments. Consequently, these results enable more targeted improvements, faster prioritization, and clearer accountability for teams.

#### 4.4 XAI Outputs Feature Level Explanations

**Extractive Explanations (Integrated Gradients).** Integrated Gradients (IG) was applied to derive token and phrase level evidence for each (review, aspect) predictions. Table 12 presents a cleaned subset of explanations sampled from the same government app reviews shown earlier in Table 11. To maintain coherence with the previous section while ensuring readability, only the app name, aspect, predicted sentiment, top-k supporting phrases, and top-k opposing phrases are retained.

**Table 12.** Extractive IG Explanations showing top-k supporting and opposing phrases and sentiment predictions for government app reviews.

| App Name                     | Aspect               | Predicted Sentiment | Top-k Supporting Phrases                        | Top-k Opposing Phrases                      |
|------------------------------|----------------------|---------------------|-------------------------------------------------|---------------------------------------------|
| BARD Mobile                  | attitude-of-staff    | negative            | never; password; never reach anyone; help       | and; great; with my Guide                   |
| Weather-CAN                  | competitor           | positive            | the best; App; love; accurate                   | weather Canada; more; that other weather    |
| NIOSH SLM app                | ease-of-use          | positive            | really; praise; clean and easy; accurate        | app developers; ads; beg; distract; noise   |
| MyHealth Records             | account-access       | negative            | Nice; access; all MY health records             | to have; to all MY                          |
| PnemoRecs VaxAdvisor         | ease-of-use          | negative            | fix add passport; not taking; passport          | Please; using camera. Is; my                |
| Mobile Passport Control      | competitor           | negative            | Terminal; useless; without; not; other airports | app; everything ready; and; clearance; Hope |
| Alternative Fueling Stations | general-satisfaction | negative            | Doesn't work in Canada                          | Doesn't work in Canada                      |
| Job Bank - Job Search        | ease-of-use          | positive            | lifesaver; easy; use; thank                     | app is a; to; even;                         |
| FEMA                         | ease-of-use          | positive            | Excellent easy; navigate,love; Thanks           | to; it; Fema                                |

The IG explanations reveal several desirable behaviors. First, highly sentiment bearing tokens such as “never”, “useless”, “lifesaver”, “accurate” and more appear among the top supporting cues, indicating that the attribution method successfully identifies linguistically meaningful evidence. Second, opposing cues often capture contextual or neutral fragments such as “is”, “it”, “using”, “weather” and more which aligns with the expectation that not all tokens contribute positively to the predicted sentiment. In cases of extremely short reviews, for example “Doesn’t work in Canada”, IG correctly highlights the entire phrase, reflecting a strong localization of sentiment.

For public-sector application teams, these extractive explanations provide actionable insights into why specific sentiments were assigned. Negative cues such as “never reach anyone” or “useless” highlight operational or workflow pain points, helping teams prioritize fixes. Positive cues such as “clean”, “easy”, and “lifesaver” or “accurate” highlight design successes that are worth preserving. More importantly, the method offers transparency where developers can trace predictions back to tangible text spans rather than treating the model as a black box, enabling greater trust and more efficient integration of sentiment analysis into improvement cycles.

**LLM-Generated Justifications (Gemini).** To complement the extractive phrase-level attributions generated by Integrated Gradients, Gemini was used to produce natural-language justifications for each (review, aspect) prediction. These justifications translate attribution cues into coherent explanations that can be more readily interpreted by teams and non-technical stakeholders. To maintain alignment with the earlier examples while preserving readability, Table 13 only retains the app name and Gemini justification. The different full version sample of reviews can be found in Appendix B.

**Table 13.** Gemini-Generated natural-language justifications grounded in the IG attribution cues for the same set of government app reviews

| App Name                | Gemini Justification                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
|-------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| BARD Mobile             | The model predicted negative sentiment for "attitude-of-staff" primarily because the user stated they "can never reach anyone to help." The word "never" strongly indicates a lack of staff availability and support, which directly relates to a negative experience with staff. Although the review includes positive phrases like "great for long routes with my Guide," these cues refer to the app's potential utility for navigation, not the staff's attitude or helpfulness.                                          |
| WeatherCAN              | The model predicted positive sentiment for the 'competitor' aspect by noting highly positive terms such as "the best," "love," and "accurate" within the review. While the phrases "weather Canada" being "more" accurate than "that other weather" highlight a comparative advantage for the main app, these opposing cues were not strong enough to override the dominant positive sentiment identified in the supporting phrases for the 'competitor' aspect.                                                              |
| NIOSH SLM               | The model predicted positive sentiment for ease-of-use primarily due to the direct positive phrase "clean and easy," reinforced by amplifiers like "really" and "praise." The term "accurate" also contributed to this positive assessment. While words such as "ads," "beg," "distract," "app developers," and "noise" were identified as minor opposing cues, these did not outweigh the strong positive evidence for the app's usability.                                                                                  |
| MyHealth Records        | The model predicted positive sentiment for 'account-access'. This is primarily driven by strong positive indicators such as "Nice" and the explicit mention of "access to all MY health records," showing satisfaction with reaching personal data. While phrases like "to have" and "to all MY" were flagged as potentially opposing, their impact is minimal, acting more as functional parts of the sentence rather than conveying negative sentiment.                                                                     |
| PneumoRecs VaxAdvisor   | The sentiment is predicted as negative for ease-of-use because the user reports a critical functional issue: the app is "not taking" their "passport" and asks to "fix add passport." This clearly indicates difficulty or failure in using a core feature. While the user included "Please" in their request and mentioned "using camera," these elements do not counteract the reported problem with the app's functionality.                                                                                               |
| Mobile Passport Control | The model predicted negative sentiment for the "competitor" aspect. This is primarily driven by the app being described as "useless" and the direct comparison to "passengers without this app." Phrases like being in the "same line" and waiting the "same amount of time as passengers without the app" highlight a clear lack of competitive advantage over the alternative. The user's expression of "Hope" this isn't true for "other airports" offers a slight nuance, indicating a desire for future differentiation. |

|                              |                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Alternative Fueling Stations | The model predicted negative sentiment for general satisfaction because the phrase "Doesn't work in Canada" clearly indicates a critical functional problem with the app. This direct statement of non-functionality strongly suggests a negative user experience. While "Doesn't work in Canada" also appears as an opposing cue, there are no distinct positive signals to provide nuance or suggest a mixed sentiment, thus reinforcing the negative prediction. |
| Job Bank - Job Search        | The model predicted positive sentiment for ease-of-use, strongly supported by cues like "lifesaver" and the direct statement "easy to use." The user's "thank" further reinforces this positive experience. While neutral connecting words such as "app is a," "to," "even," and "you" were flagged as opposing cues, they do not introduce any negative sentiment or contradict the product's ease of use.                                                         |

The Gemini-generated justifications align closely with the extractive explanations produced by Integrated Gradients, accurately emphasizing the sentiment related phrases identified by IG. In most cases, Gemini correctly focuses on the same high impact cues such as “never reach anyone”, “clean and easy” or “lifesaver” and ignores semantically weak or neutral opposing cues. The justifications also demonstrate strong sensitivity to contextual meaning, for example the understanding that the praise for navigation supports ease-of-use. Minor inconsistencies appear where short reviews lack nuance, for example “Doesn’t work in Canada”, through Gemini’s outputs remain faithful to the sentiment expressed.

These natural-language justifications substantially enhance interpretability, enabling team to further reinforce their understanding of the why behind the model’s predictions. By grounding explanations in specific issues, such as login failures, non-functional features, or lack of competitive advantage, the justifications help identify actionable failure points and user pain areas. Moreover, the consistency between IG cues and Gemini narratives increases trust in the pipeline, supporting its use in operational decision-making and continuous improvement cycles.

#### 4.5 Explainability Metrics

To evaluate the reliability and behavioral quality of the explanations produced by the IG Pipeline, the comprehensiveness, sufficiency, sparsity, local sensitivity, monotonicity, and polarity alignment metrics were calculated and aggregated over all predictions with explanations. These scores quantify the degree to which explanations meaningfully influence the model’s predictive behavior and align with its internal attribution patterns. Table 14 summarizes the resulting metric values.

It is important to note that there are no universally accepted numerical benchmarks or threshold values for attribution-based explanation metrics such as comprehensiveness, sufficiency, monotonicity, sparsity, or local sensitivity. These metrics are designed to be interpreted relatively and behaviorally, rather than against fixed values, with validity assessed through the expected directional trends, internal consistency, and the avoidance of extremes. Consequently, the evaluation focuses on whether explanations exhibit theoretically grounded behavior under controlled perturbations and masking operations, rather than whether they exceed predefined numeric values.

**Table 14.** Aggregated explainability metrics capturing fidelity, robustness, and polarity alignment of the model explanations generated by the IG pipeline

| Metric            | Score    |
|-------------------|----------|
| Comprehensiveness | 0.339067 |
| Sufficiency       | 0.396375 |
| Sparsity          | 0.207711 |
| Local Sensitivity | 0.00008  |
| Monotonicity      | 0.755416 |
| Support Polarity  | 0.178843 |
| Oppose Polarity   | 0.165421 |

Across metrics, the explanation set exhibits reasonably strong behavioral fidelity. The monotonicity score (0.755416) is the strongest metric, indicating that removing negatively attribute tokens tends to increase confidence in the predicted class, and removing positively attributed tokens tends to decrease it which is consistent with the expected behavior of a faithful attribution method. The sufficiency (0.396375) and comprehensiveness (0.339067) values further suggest that the highlighted tokens contribute significantly to the model’s decisions, while no small subset alone fully preserves or eliminates the prediction. This pattern is consistent with prior observations in transformer-based models, typically depend on distributed representations spanning multiple tokens rather than isolated cues. The sparsity score (0.207711) shows that attributions are relatively concentrated on a small subset of tokens rather than diffused across the entire input, supporting interpretability. Meanwhile, local sensitivity values were near zero under small Gaussian perturbations ( $\sigma = 10^{-4}$ ), indicating that IG attributions are numerically stable to infinitesimal noise. As expected, the magnitude of this metric is bounded by the perturbation scale and therefore reflects continuity under tiny perturbations rather than behavioral robustness. The support and oppose polarity values (0.178843 and 0.165421) further shows that positively attributed tokens tend to be more positive in sentiment polarity, and negatively attributed tokens tend to be more negative, suggesting coherent alignment between the sentiment of the language and attribution direction.

These metrics combined point toward an explanation framework that is both stable and behaviorally informative. Strong monotonicity strengthens trust that the highlighted cues correspond to genuine evidence that the model relies upon. The moderate comprehensiveness and sufficiency indicate that the explanations capture important but not exhaustive evidence, which is desirable when presenting concise cues to teams. Low sensitivity and consistent polarity alignment suggests that the explanations are robust to small changes and are linguistically coherent. This combination is particularly valuable because explanation stability and clarity support responsible decision making and aid the identification of actionable pain points.



## 5 Discussion

The results presented in this study directly address the central research problem defined in the introduction, the lack of automated, fine-grained, and interpretable sentiment analysis frameworks suitable for large-scale analysis of government mobile application reviews. The strong predictive performance achieved on both the FABSA benchmark, and the collection of real-world government app reviews demonstrates that the transformer-based ABSA model can successfully generalize across domains. In doing so, the first research question concerning the feasibility of adapting modern ABSA models to public-sector app feedback is answered. The observed micro-F1 and macro-F1 scores align with, and in some cases exceed, those reported in previous evaluations (Chaudhry et al., 2025), confirming that fine-tuning on aspect-level supervision enables robust sentiment differentiation under a substantial class imbalance.

Beyond predictive accuracy, the results significantly engage with the second research question, which concerns the interpretability of ABSA outputs in a public-sector context. Integrated Gradients consistently produced attribution patterns that were linguistically coherent and aligned with the sentiment, with high monotonicity and near-zero local sensitivity indicating stable and behaviorally faithful explanations. These findings directly support the objective of pairing aspect-level sentiment predictions with explanation mechanisms that expose the textual evidence underlying model decisions. As a result, this study responds to the limitations identified in the literature, where prior ABSA systems either lacked interpretability altogether or relied on explanation techniques whose reliability was not quantitatively validated.

The alignment between IG-based extractive explanations and Gemini-generated natural-language justifications further addressed the third research question regarding the corresponding role of large language models in explanation generation. Rather than introducing novel reasoning or unsupported claims, Gemini’s justifications closely mirrored the attribution cues identified by IG, suggesting that LLMs can be used effectively as translation layers, that render technical attribution into accessible rationales, when grounded in model-internal evidence. This outcome reinforces concern raised in earlier studies about explanation hallucination, while simultaneously demonstrating a viable strategy that remains consistent with existing interpretability theory.

The quantitative explainability metrics reported in Section 4.5 provide additional confirmation that the explanations behave as theoretically expected. Moderate comprehensiveness and sufficiency scores reflected the distributed nature of transformer representations, as discussed in prior XAI literature, rather than a failure of the attribution method. Similarly, the low sparsity values indicate that sentiment decisions are supported by compact yet semantically meaningful subsets of the input. This pattern, characterized by high monotonicity, low local sensitivity, and compact attribution sets, aligns with the theoretical expectations motivating gradient-based explanation methods for transformer models, where stable and monotonic attribution behavior is taken as an indicator of explanation faithfulness.

Taken together the findings substantiate the contributions outlined in the introduction. First, the study demonstrates that FABSA-trained ABSA models can be success-

fully extended to the public-sector context without task-specific reannotation, addressing the gap between commercial and public-sector sentiment analysis identified in the literature. Second, it introduces an end-to-end interpretability framework that integrated attribution-based explanations, LLM generated rationales, and quantitative evaluation, therefore advancing methodological practice in explainable sentiment analysis. Finally, by grounding all interpretability claims in empirical metrics and observable model behavior, the work contributes to ongoing efforts within the field to move beyond unbiased explanations toward more accountable and trustworthy NLP systems.

Concisely, the results confirm that the proposed framework meets its stated objectives. It delivers accurate aspect-based sentiment predictions, produces stable and faithful explanations, and supports scalable analysis of government mobile application feedback. By resolving the research questions and aligning closely with the challenges identified prior work, this study offers a coherent and empirically grounded contribution to the intersection of ABSA, explainable AI, and public-sector applications.

## 6 Conclusion and Future Work

This study aimed to address a critical gap at the intersection of aspect-based sentiment analysis, explainable AI, and public sector digital services. As governments increasingly rely on mobile applications to deliver essential services, user reviews represent a rich but underutilized source of evidence for understanding public experience. However, existing sentiment analysis approaches either lack the granularity required to isolate specific service issues or fail to provide explanations that are sufficiently transparent for critical public-sector decision-making. Motivated by these challenges, this work proposed and evaluated an end-to-end framework for fine-grained, interpretable sentiment analysis of government mobile application reviews.

The study demonstrated that a transformer-based ABSA model fine-tuned on the FABSA dataset generalizes effectively to real-world North American government app reviews, despite domain shift and substantial class imbalance. Strong performance on the FABSA test set, reflected in high micro and macro averaged metrics and substantial Cohen’s Kappa agreement, confirms that aspect-level supervision enables robust sentiment differentiation beyond coarse document-level analysis. When applied to public-sector applications, the model produced coherent and plausible aspect-sentiment assignments that aligned closely with known user pain points such as login failures, usability breakdowns, and lack of staff support. These findings directly address the first research question and fulfill the objective of evaluating FABSA-trained models under realistic public-sector conditions.

Beyond predictive accuracy, this study treated interpretability as a core principle rather than a secondary consideration. Integrated Gradients provided stable and linguistically meaningful token and phrase-level attributions, while the incorporation of large language model-generated justification translated these technical explanations into accessible narratives for non-technical stakeholders. Quantitative explainability metrics further showed that explanations exhibit desirable behavioral properties, including high

monotonicity, low local sensitivity, and coherent polarity alignment. Collectively, these results answer the second and third research questions by demonstrating that attribution-grounded explanations can be both faithful to the model’s internal reasoning and interpretable at human level. As a result, the study advances the state of explainable ABSA by grounding interpretability claims in measurable, model-behavioral evidence rather than intuition alone.

The implications of these findings reinforce the central argument articulated in the introduction, public-sector digital services require analytical tools that not only scale to large volumes of user feedback but also support trust, accountability, and actionable insight. By enabling aspect-level diagnosis and transparent explanation, the proposed framework offers a practical pathway for integrating sentiment analytics into public-sector improvement workflows. At the same time, the work contributes methodologically to the broader ABSA and XAI literature by demonstrating how gradient-based attributions, LLM rationales, and quantitative evaluation can be combined into a coherent and reproducible pipeline.

Despite these contributions, several limitations should be acknowledged. First, the ABSA model was trained exclusively on the FABSA dataset, which while diverse, does not fully capture the linguistic and institutional specificity of government services. Second, explainability metrics were adapted from general XAI frameworks, and although informative, may not exhaustively capture all dimensions of explanation usefulness for public-sector stakeholders. Third, LLM-generated justifications were evaluated qualitatively rather than through formal human-subject studies, leaving open questions about their effectiveness in real decision-making contexts.

While the proposed framework demonstrates strong performance and interpretability under realistic public-sector condition, these limitations point toward several promising directions for future investigation. First, model generalization under more severe domain shift could be further strengthened by expanding training with additional domain-specific datasets, particularly those derived from government digital services. Broader coverage of institutional language and region-specific service contexts may improve robustness when deploying ABSA systems across similar public platforms. Second, recent advances in large language models open opportunities to explore alternative modelling ideas for aspect-based sentiment analysis. Incorporating instruction-tuned or few-shot LLMs for ABSA would allow systematic comparison between hybrid pipelines, such as the one proposed in this study, and fully end-to-end LLM-based approaches. Such comparisons could clarify trade-offs between performance, interpretability, computational cost, and controllability, particularly in public-sector settings where reproducibility and transparency remain critical constraints. Beyond end-to-end language models, future work could also explore hybrid architectures that combine transformer encoders with structured relational models, such as graph neural networks (GNNs). For example, pairing DistillRoBERTa with a GNN over dependency graphs or aspect-opinion interaction graphs may improve modelling of cross-aspect relationships and long-range dependencies, while retaining the strong contextual representations of pretrained transformers. These hybrid designs have shown promise in recent ABSA research (Alhadlaq et al., 2024) and may offer a complementary trade-off between expressiveness, interpretability, and computational efficiency. Third, although

Integrated Gradients provided stable and interpretable attributions in this study, future work may benefit from exploring robust attribution techniques and multi-explanation ensembles. Recent research on explanation uncertainty suggests that optimized aggregation of feature attributions can improve explanation faithfulness and reliability (Decker et al., 2024), especially for transformer models operating on noisy natural language. Leveraging insights from emerging work on explanation consistency could further strengthen confidence in explanation outputs. Finally, a natural extension of this study is the deployment of the framework as an interactive dashboard for real-time monitoring of public digital service quality. By continuously ingesting app reviews, generating aspect-level sentiment trends, and surfacing interpretable explanations, such a system could support evidence-based prioritization of development resources. This direction directly reflects the broader motivation of the study moving from retrospective analysis toward actionable, transparent, and operational sentiment intelligence in digital government ecosystems.

In conclusion, this study presents an empirically grounded framework that unifies fine-grained sentiment modelling and explainable AI for public-sector mobile application. By directly addressing the research questions, meeting the stated objectives, and delivering the contributions outlined in the introduction, the work offers a coherent and practical contribution to the growing field of interpretable sentiment analysis in digital government contexts.

## 7 References

1. Alhadlaq, A., & Altheneyan, A. (2024). Distilroberta2gmn: a new hybrid deep learning approach for aspect-based sentiment analysis. *PeerJ Computer Science*, 10, e2267. <https://doi.org/10.7717/peerj-cs.2267>
2. Chaudhry, H. N., Kulsoom, F., Ullah Khan, Z., Aman, M., Khan, S. U., & Albanyan, A. (2025). TAsCI: transformers for aspect-based sentiment analysis with contextual intent integration. *PeerJ Computer Science*, 11, e2760. <https://doi.org/10.7717/peerj-cs.2760>
3. Decker, T., Bhattarai, A. R., Gu, J., Tresp, V., & Buettner, F. (2024). Provably Better Explanations with Optimized Aggregation of Feature Attributions. *ArXiv (Cornell University)*. <https://doi.org/10.48550/arxiv.2406.05090>
4. Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2018). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. *ArXiv.org*. <https://arxiv.org/abs/1810.04805>
5. DeYoung, J., Jain, S., Rajani, N. F., Lehman, E., Xiong, C., Socher, R., & Wallace, B. C. (2019). ERASER: A Benchmark to Evaluate Rationalized NLP Models. *ArXiv.org*. <https://arxiv.org/abs/1911.03429>
6. Distilroberta Base. (2023). *Dataloop.ai*. [https://dataloop.ai/library/model/distilbert\\_distilroberta-base/](https://dataloop.ai/library/model/distilbert_distilroberta-base/)

7. E-Government Development Index (EGDI) by region 2024 Statista. (2024). Statista. <https://www.statista.com/statistics/421584/egdi-e-government-development-index-region/>
8. Géron, A. (2019). Hands-on machine learning with Scikit-Learn and TensorFlow concepts, tools, and techniques to build intelligent systems (2nd ed.). O'Reilly Media, Inc.
9. Highest-rated U.S. government websites 2020 Statista. (2021). Statista. <https://www.statista.com/statistics/265835/egovernment-pages-ranking/>
10. Jurafsky, D., & Martin, J. (2024). Speech and Language Processing An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition with Language Models Third Edition draft Summary of Contents. [https://web.stanford.edu/~jurafsky/slp3/ed3bookaug20\\_2024.pdf](https://web.stanford.edu/~jurafsky/slp3/ed3bookaug20_2024.pdf)
11. Kontonatsios, G., Clive, J., Harrison, G., Metcalfe, T., Sliwiak, P., Tahir, H., & Ghose, A. (2023). FABSA: An aspect-based sentiment analysis dataset of user reviews. *Neurocomputing*, 562, 126867. <https://doi.org/10.1016/j.neucom.2023.126867>
12. Martens, D., & Johann, T. (2017). On the Emotion of Users in App Reviews. Retrieved December 3, 2025, from <https://arxiv.org/pdf/1703.02256>
13. McHugh ML. (2025). Interrater reliability: the kappa statistic. *Biochemia Medica*, 22(3). <https://pubmed.ncbi.nlm.nih.gov/23092060/>
14. Nortal. (2024, April 24). 70% of Canadians Demand Fully Digital Public Services & 87% Expect This By 2026. Nortal.com; Nortal. <https://nortal.com/news/news-library/70-of-canadians-demand-fully-digital-public-services-87-expect-this-by-2026/>
15. Seth, P., Rathore, Y., Singh, N., Chitroda, C., & Kumar Sankarapu, V. (2025). xai\_evals : A Framework for Evaluating Post-Hoc Local Explanation Methods. In *arxiv.org*. <https://arxiv.org/html/2502.03014v1>
16. Shah A. F., Sabir A., & Sharma R. (2024, September 11). A Fine-grained Sentiment Analysis of App Reviews using Large Language Models: An Evaluation Study. Retrieved December 3, 2025, from <https://arxiv.org/pdf/2409.07162v1>
17. U.S. customer satisfaction with e-government 2020 Statista. (2021). Statista. <https://www.statista.com/statistics/184361/us-consumer-satisfaction-egovernment/>
18. U.S. customer satisfaction with mobile e-government 2020 Statista. (2021). Statista. <https://www.statista.com/statistics/269971/us-consumer-satisfaction-mobile-egovernment/>
19. Ullah, W., Zhang, Z., & Stefanidis, K. (2023). Sentiment Analysis of Mobile Apps Using BERT. *Lecture Notes in Computer Science*, 66–78. [https://doi.org/10.1007/978-3-031-36822-6\\_6](https://doi.org/10.1007/978-3-031-36822-6_6)
20. Walker, C., Jha, S., Chen, K., & Rickard Ewetz. (2024). Integrated Decision Gradients: Compute Your Attributions Where the Model Makes Its Decision. *Proceedings of the AAAI Conference on Artificial Intelligence*, 38(6), 5289–5297. <https://doi.org/10.1609/aaai.v38i6.28336>

## Appendix A

### Gemini Natural Language Justification Prompt

```

text = shorten(str(row["text"]), width=600, placeholder=" ...")
aspect = str(row["aspect"]) if not pd.isna(row["aspect"]) else None
label = str(row.get("pred_label_str", row.get("pred_label_xai", row["pred_label_id"])))
sup = str(row.get("evidence_support_topk", "")).strip()
opp = str(row.get("evidence_oppose_topk", "")).strip()

aspect_part = f" for the aspect **{aspect}**" if aspect else ""
support_part = sup if sup else "N/A"
oppose_part = opp if opp else "N/A"

prompt = f"""
You are an explainable AI assistant.

A sentiment model predicted **{label}** sentiment{aspect_part} for the following
user review of an app. The review text and the key evidence phrases (from an
attribution method) are provided.

Your job:
- Write a short justification (2 to 4 sentences, less than or equal to 80 words).
- Base your reasoning only on the evidence phrases given.
- Use the opposing cues (if any) to mention nuances or mixed signals.
- Do NOT invent new evidence or speculate about user intent beyond what is shown.

Review text:
  \"{text}\"

Supporting cues (evidence for the predicted sentiment): {support_part}

Opposing cues (evidence against the predicted sentiment): {oppose_part}

Now write the justification in plain English, as if explaining the model's
decision to a product team.
"""

```

## Appendix B

Full version of Sentiment Prediction, XAI and Gemini sample that Table 11, 12 and 13 were drawn from can be found at the following link:

Full Version Prediction XAI and Gemini Sample.xlsx